



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第八章 极化码

李丽香，彭海朋

北京邮电大学网络安全学院

网络与交换技术国家重点实验室



极化码（Polar Code）是由土耳其 Bilkent 大学的 Erdal Arıkan 教授于 2008 年发明的一种信道编码方法。Polar 码是目前唯一的一种能够达到香农极限（Shannon Limit）的编码方法，而比较其他编码方法只是接近香农极限。也是目前国际移动通信标准化组织认定的 5G 场景控制信道的编码方案。



极化码优点：

- 1、是目前唯一的香农信道容量可达的编码方式
- 2、有坚实的理论基础
- 3、编解码复杂度低
- 4、精细的码率调整机制（信息块长度可以一比特一比特的递减）
- 5、极化码的递归特性易于通过硬件实现（母码长度为 N 的Polar码可以用两个母码长度为 $N/2$ 的Polar码实现）

极化码缺点：

- 1、汉明距离较小，在短码情况下影响解码性能，一定程度上可以通过选择合适的冻结比特位置进行规避。适合长码编码。
- 2、SC译码延时较大。



2009年

日本京都大学的教授使用密度演进的方法研究了Polar码的性能。

2013年

美国德州大学的H. Si研究了在衰落信道下的分层Polar码，其编码方案达到了衰落DMC下的性能。

2014年

加拿大的P.Giard研究了在普通CPU上实现的Polar码软件解码算法⁷，其吞吐量超过了200Mbit/s。

2017年

在美国Spokane的3GPP RAN WG1#AH1_ NR会议上，不具有比特翻转模块的Polar码得以确定在5G中使用

2016年

美国Reno的3GPP TSG RAN WG1#87会议上, Polar码被确定为第五代移动通信(5G)的增强移动宽带(eMBB)物理控制信道的编码方案。

2016年

在3GPP第71次RAN全会上，下一代新的无线接入网(NR、5G、5G-NR)的研究项目的立项通过。Polar码在短码长、低码率上的性能和实现复杂度优势。

- 在国内，北京邮电大学是最早研究Polar码的机构之一。牛凯教授的团队研究了Polar码的编码和解码算法等。
- 西安电子科技大学李颖教授的团队研究了Polar码的高斯近似的码构造方法和解码算法。
- 哈尔滨工业大学王学东教授的团队研究了Polar 码的编码与译码算法。

截止频率越高，系统响应越快

因此Polar码的最初设计的出发点是为了提升截止频率

在随机编码和最大似然条件下，信道截止速率 R_0 决定了相应的码块的错误概率：

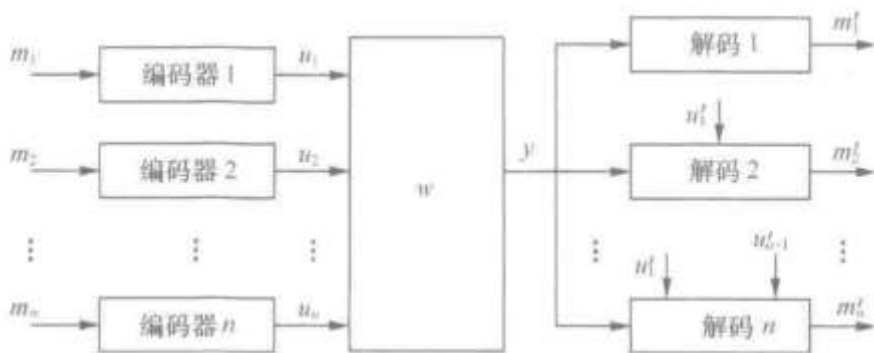
$$P_e = 2^{-N \cdot R_0}$$

其中， N 为码块长度。实际信道的传输速率 R 小于 R_0 、使用随机编码和ML编码时，信道平均错误概率：

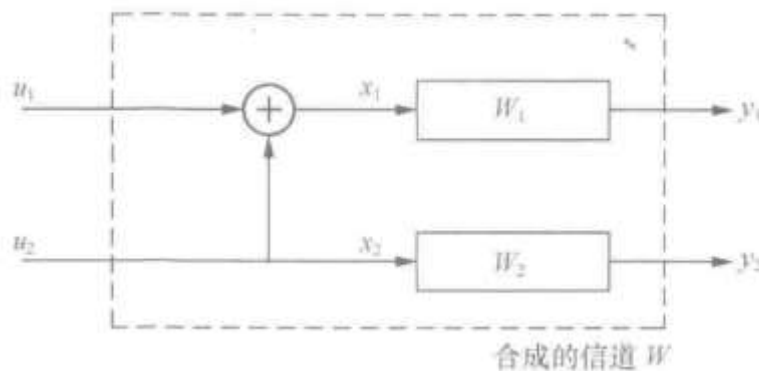
$$\bar{P}_e = 2^{-N (R_0 - R)}$$

因此，在2004-2006年， Arikan 教授尝试使用信道合并与分离的方法来提升信道截止速率。

对于一个离散无记忆的信道 W ，分离出两个相关的子信道 W_1 和 W_2 ，有 $C(W_1) + C(W_2) \leq C(W)$ ，同时有 $R_0(W_1) + R_0(W_2) > R_0(W)$ 。其中 C 是信道容量， R_0 是信道截止速率，即两个子信道信道容量不超过原本的总信道容量，但分离后总信道截止速率提高了。



信道分离示意图



信道合并示意图

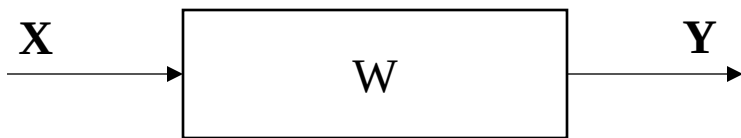
对于Polar码基本原理可以归结为三点：信道合并、信道分离、信道极化。其中信道合并和信道极化是在编码时完成的；信道分离是在解码时完成。

信道选择：

对于编码方案通常根据某种信道设计，Polar码主要考虑以下信道进行分析：



信道 W : $X \rightarrow Y$ 。输入变量 X 只能取 $\{0,1\}$ 中的值，输出变量 Y 取任意值，信道迁移概率为 $P=W(y|x)$, $x \in X$, $y \in Y$ 。



信道示意

对于对称信道（BEC\BSC），当输入在 $\{0,1\}$ 概率时，其信道容量定义为输入输出的相互信息。

$$C(W)=I(X;Y)=\sum_{y \in Y} \sum_{x \in X} \frac{1}{2} W(y|x) \log \left(\frac{W(y|x)}{\frac{1}{2} W(y|0) + \frac{1}{2} W(y|1)} \right)$$

信道合并（编码时）：

将两个BEC信道 W_1 和 W_2 合并成一个信道 W 。其中，“+”为模2加。由于 u_1 和 u_2 都取自 $\{0,1\}$ 且独立同分布，那么信道容量为：

$$C(W)=I(U;Y)$$

其中， $U=[u_1,u_2]$, $Y=[y_1,y_2]$ 。

$$C(W)=I(X;Y)$$

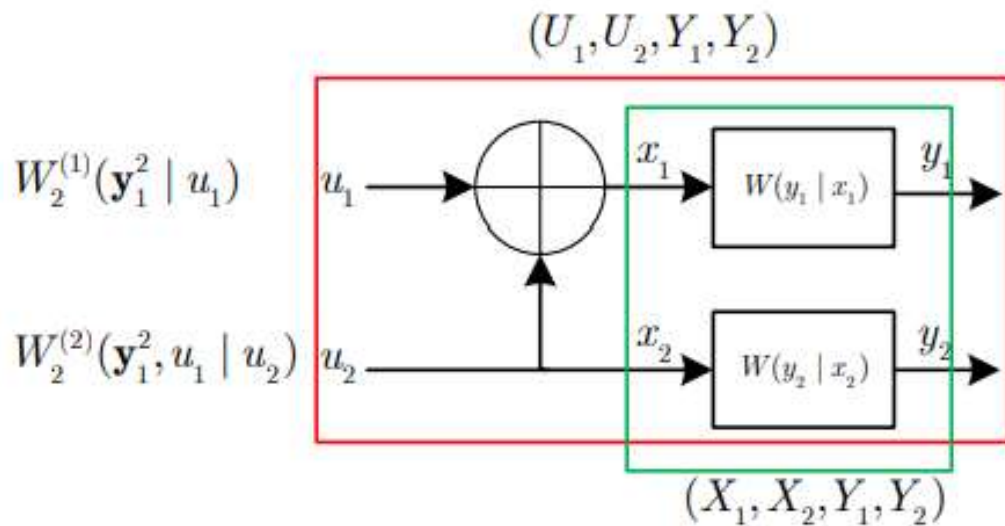
其中， $X=[x_1,x_2]$ 。

$$\begin{aligned} C(W) &= I(x_1;y_1) + I(x_2;y_2) = C(W_1) + C(W_2) \\ &= 2 \times C(W_1) = 2 \times C(W_2) \end{aligned}$$

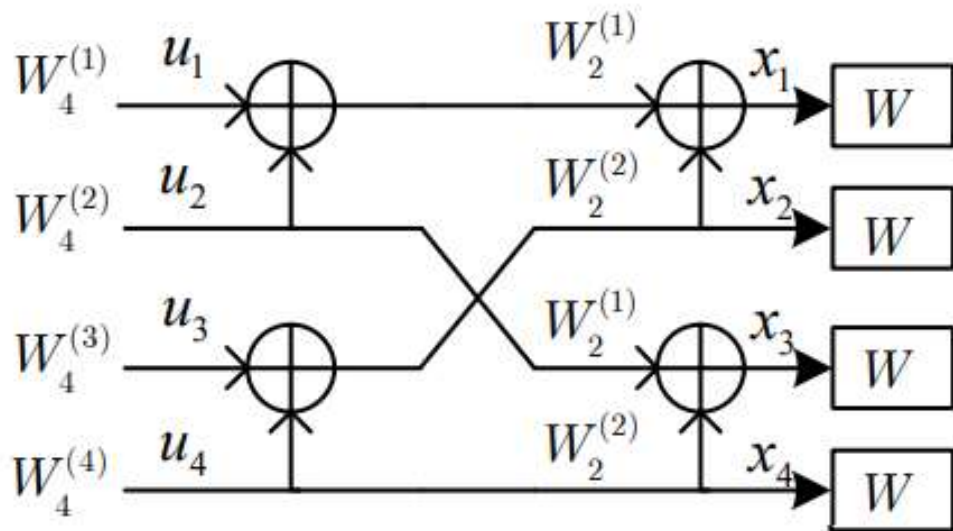
也就是说，信道合并保持总信道容量守恒。若 N 个信道合并，则

$$C(W)=N \cdot C(W_1)$$

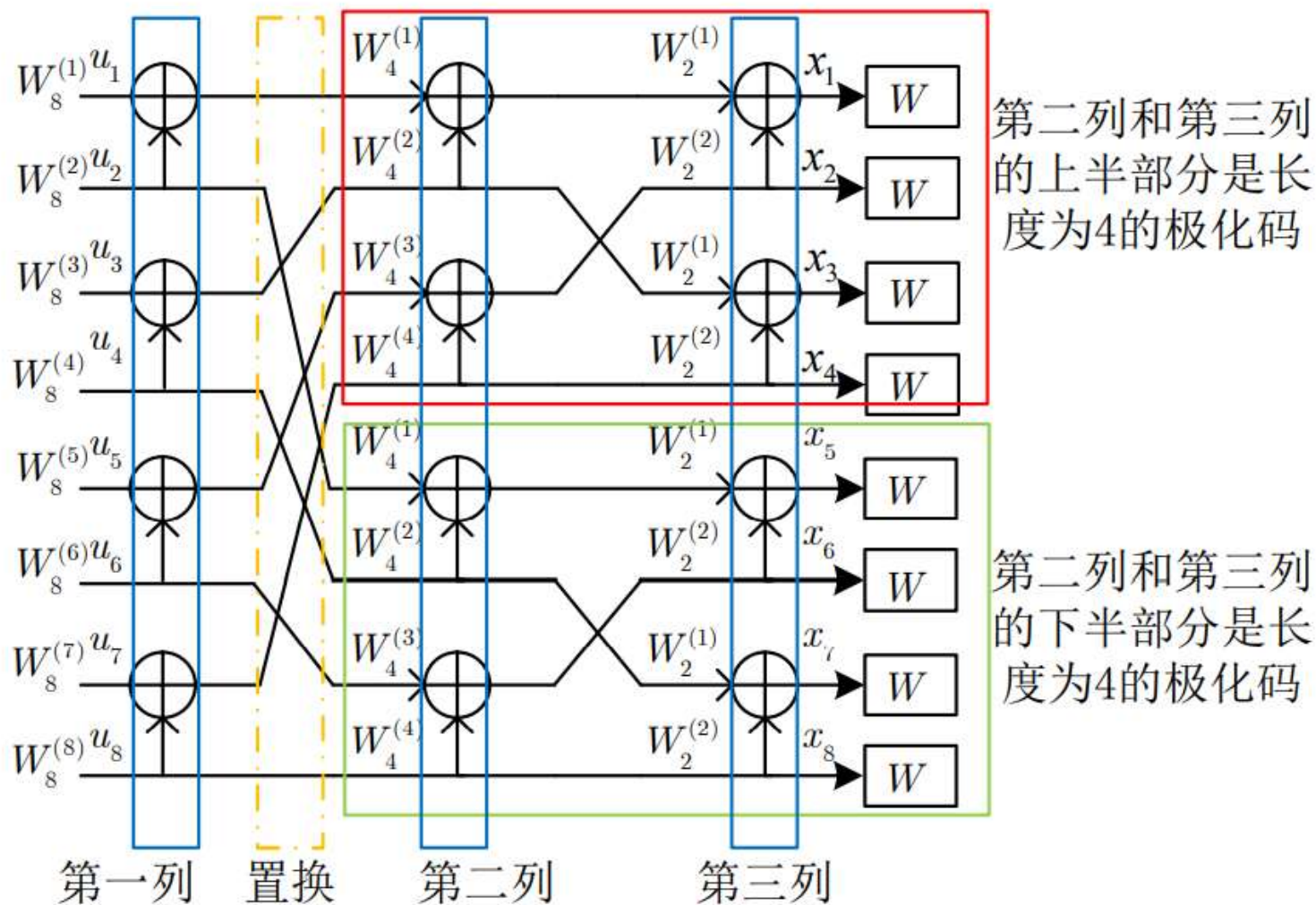
即，合成信道的总容量是各个子信道容量之和，容量守恒。



双信道合并示意



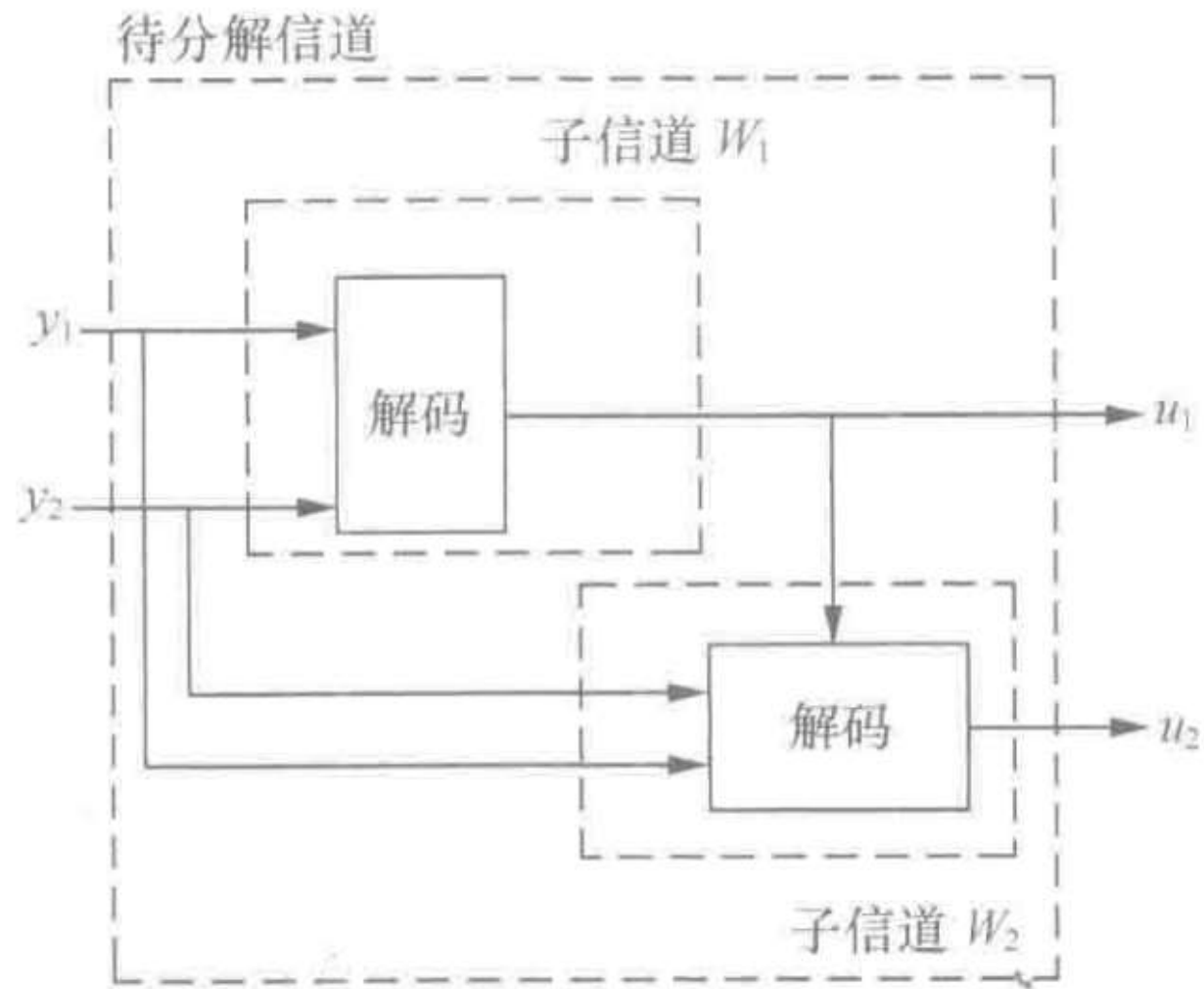
4个BEC信道合并示意



多信道合并示意

信道分离（解码时）：

信道分离是将混有多个输入比特信息的接收信息 $y_1^N = (y_1, y_2, y_3, \dots, y_N)$ 分解成独立的多个输入比特信息 $u_1^N = (u_1, u_2, u_3, \dots, u_N)$ 。考虑针对G2 Polar 码编码而成的信道解码过程。首先解码器从 $y^N = y^2 = (y_1, y_2)$ 分解出第一个子信道 u_1 。之后再从 (u_1, y_1, y_2) 中分解出第二个子信道 u_2 ，从而完成了 $Y^N \rightarrow U^N$ ， $N=2$ 的信道分解过程。信道分离过程也是容量守恒的。



信道分离示意

信道极化（编码时）：

信道极化是指随着Polar码母码长度 $N=2^n$ 的增大，子信道（Polar码输入比特序号）的信道容量会越来越明显的趋向两个极端：无噪信道（信道容量为1）和纯噪信道（信道容量为0）。

信道极化（编码时）：

假设使用BEC信道，且删除概率为 $P = 1/2$ ，那么随着Polar码母码长增大，子信道趋近于两个极端，当母码长度 $N \geq 512$ 时，极化现象体现较为充分（大于80%的子信道得到极化），利用这个极化现象，可以选择高容量子信道去传输数据，而把其他子信道设置为已知比特（冻结比特，如0）。

若使用以2为底的对数，则 $0 \leq C(W) \leq 1$ 。当 $C(W)=1$ 时，称 W 为无噪信道（极好）；当 $C(W)=0$ 时，称 W 为纯噪信道（无用）。信道极化的目的就是将大多数常规信道转化为这两种极端情况的信道，上述信道容量计算复杂，可采用巴氏参数的最大后验概率（MAP）判决错误概率的上界，进而进行信道容量的范围考察。巴氏参数定义为 $Z(W)$ ：

$$Z(W) = \sum_{y \in Y} \sqrt{W(y|0)W(y|1)}$$

主要通过信道容量分析极化码。

$$C(W) \geq \log \frac{2}{1+Z(W)}$$

$$C(W) \leq \sqrt{1-Z(W) \cdot Z(W)}$$

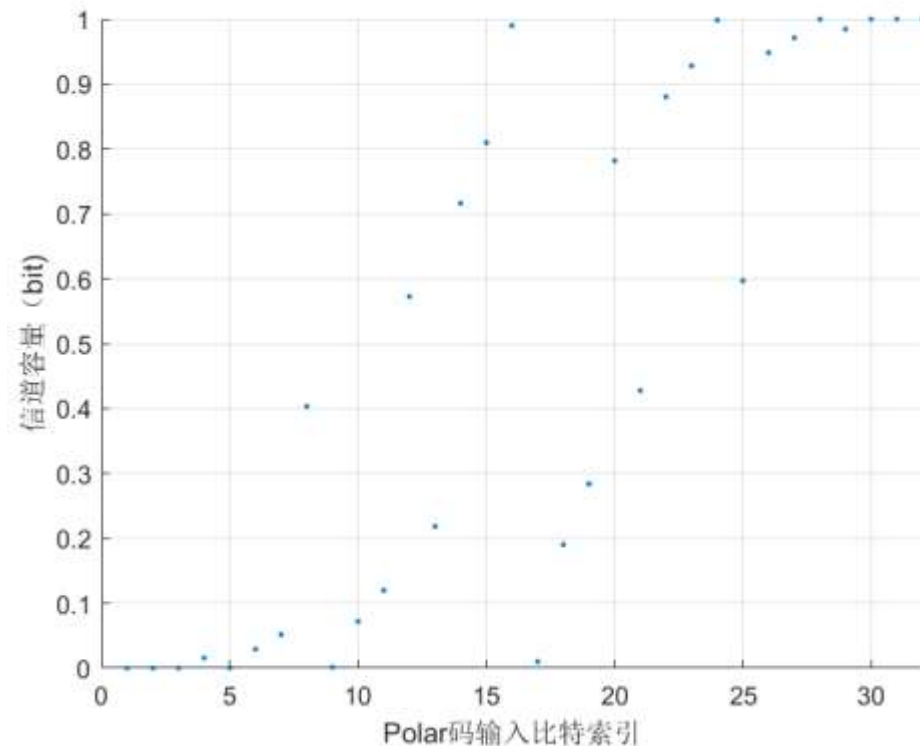
因此，针对二进制删除信道（BEC），信道容量 $\{I(W_N^{(i)})\}$ 数据可以通过递归关系计算出来。

$$I(W_N^{(2i-1)}) = I(W_{N/2}^{(i)})^2$$

$$I(W_N^{(2i)}) = 2I(W_{N/2}^{(i)}) - I(W_{N/2}^{(i)})^2$$

其中 $I(W_1^{(1)}) = 1 - \varepsilon$ ，删除概率 $\varepsilon = 0.5$ 。

信道极化的统计图显示了 $I(W^{(i)})$ 对于小的指数 i 趋向接近于“0”，对于大的指数 i 趋向接近于“1”。然而，对于中间范围的指数 i ， $I(W_N^{(i)})$ 显示了不规则的特性。图中的所有点是呈现中心对称的，这也是极化码的信道极化现象所具有的特性。

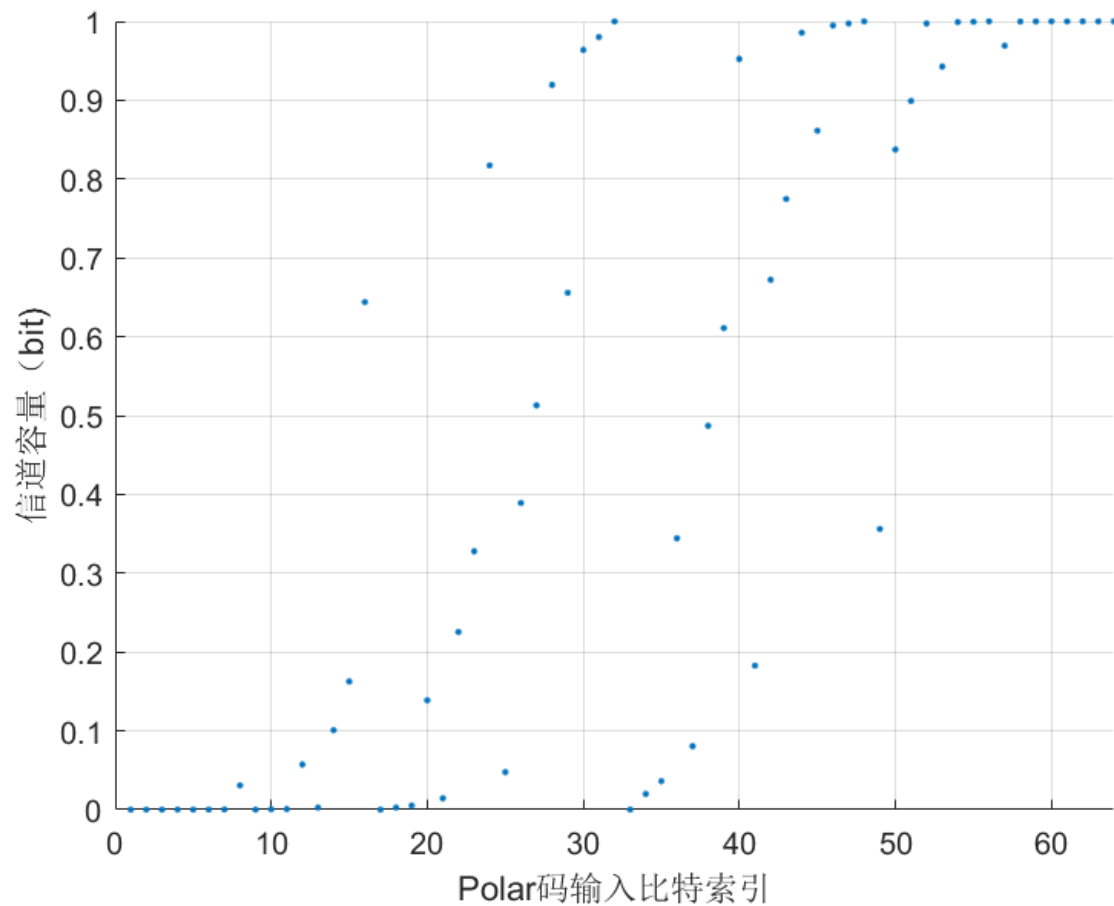


Polar码信道容量 (N=32)

码长 $N=2^5$ 的信道极化现象

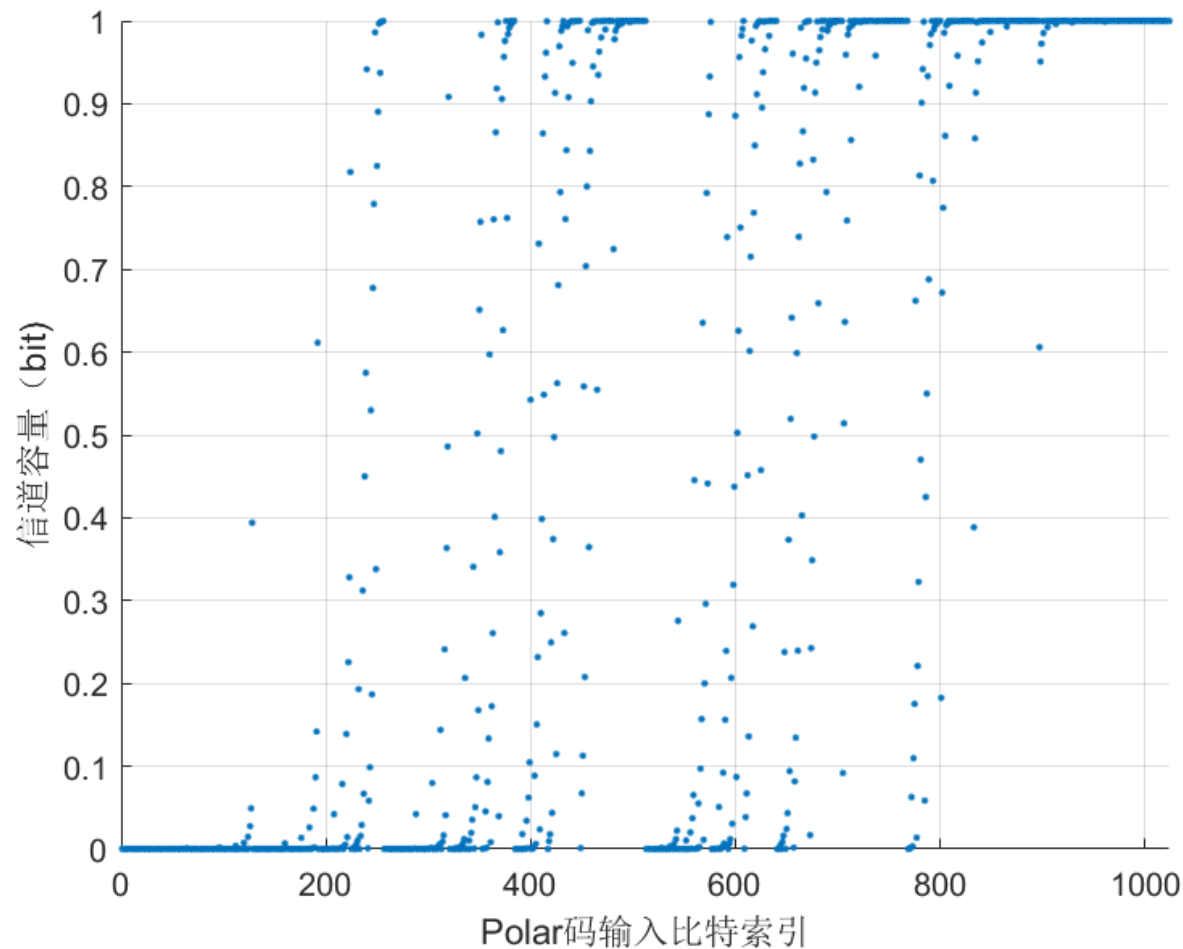
信道极化的统计图

随着子信道越来越多，信道容量就会越来越趋近于两端。



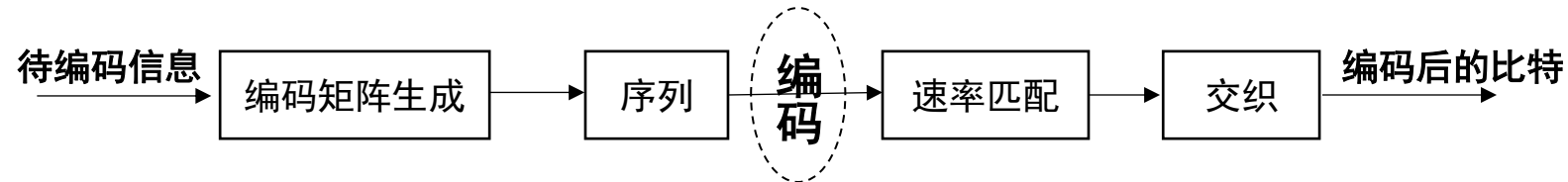
Polar码信道容量 (N=64)

码长 $N=2^6$ 的信道极化现象



码长 $N=2^{10}$ 的信道极化现象

Polar码构造是指为Polar码能够良好工作设计的框架，其中包括编码矩阵生成、序列、速率匹配、交织



Polar码构造

Polar码的编码矩阵是指在进行极化编码时用来计算编码字的矩阵（ $c=u \cdot G$ 中的 G ）编码矩阵是通常是事先存储好而不需要动态改变的。一个母码长度为 $N = 2^n$ 的极化码，其编码矩阵可由：

$$\text{Arikan核} F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

计算出来 $G_N = F^{\otimes n}$ ，其中 $\otimes$ 表示张量积。

$$\text{例如： } G_2 = F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad G_4 = F^{\otimes 2} = F \otimes F = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}。$$

Polar码序列也称为“好信道选择”。用于指示Polar码编码前的比特选择顺序。Polar码可以按照信道的可靠度从低到高排列（默认用3GPP方式），也可以按照可靠度从高到低排列。

通常使用行权重（RW）序列，对于生成矩阵G的每一行，计算出所有元素的总和得到各行的行权重，然后对行权重由低到高排序（越大可靠度越高），则各行权重对应行号为所述RW序列。

例如，对于 G_8 Polar码，其生成矩阵为：

$$G_8 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

各行的权重分别为1、2、2、4、2、4、4、8那么 G_8 的Polar码的（RW）序列为{0, 1, 2, 4, 3, 5, 6, 7}。如果有两行权重相同，则行号大的排在后面（也可排在前面）

1、在线计算

不用存储数据或只需存储少量数据即可，但是需要消耗计算量，若计算量较大会产生时延。

2、嵌套性

在母码长度为 N 的序列中，删除大于或等于 $N/2$ 的元素而保持剩下的元素顺序，得到母码长度为 $N/2$ 的序列，则称母码长度为 $N/2$ 的序列是嵌套在母码长度为 N 的序列中的。

嵌套的好处在于不需要重新计算，不需要重新排序，只需一个查找表。

3、对称性

在母码长度为 N 的序列中，前后 $N/2$ 是元素关于母码长度对称，对称性可以减少存储量。

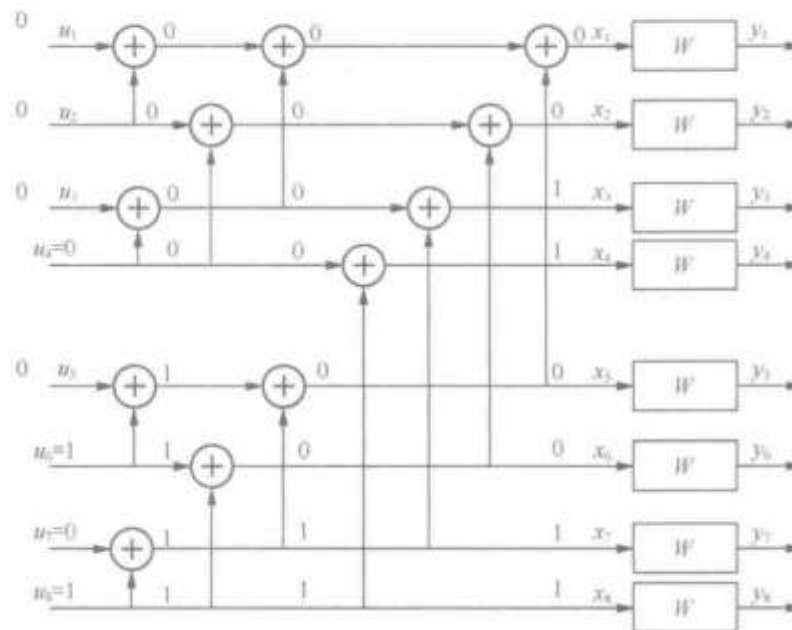
Polar码编码需选择合适的承载数据信道和放置其他信道（冻结比特）的子信道，然后进行逻辑运算的过程。通常选用承载数据的信道应选用信道容量高或可靠度高的信道。

-----根据行权重进行选取（码序列选择）

例1： 选用 $N=8$ 、 $R=1/2$ 的编码，则应选用 u_4, u_6, u_7, u_8 (0,1,0,1) 来进行编码。由于要发射的比特数不一定刚好是编码之后的比特数（如要发射12个比特），因此，在编码结束后，还要进行比特选择过程（后续讲解）。

子信道容量
(越大越好) (越小越好)

0.0039	8	冻结比特
0.1210	7	冻结比特
0.1914	6	冻结比特
0.6835	4	数据
0.3164	5	冻结比特
0.8085	3	数据
0.8789	2	数据
0.9960	1	数据



0,0,0,0,0,1,0,1(原)



0,0,0,0,0,1,1,1(一级编码)



0,0,0,0,0,0,1,1(二级编码)



0,0,1,1,0,0,1,1(三级编码)

例2：（计算）设码长 $N=8$ ，消息比特数 $k=4$ 。（部分教材极化码用 c 表示，也可用 x 表示）

需要的公式：

$$c_1^N = u_1^N G_N$$

$$G_N = B_N F^{\otimes n}$$

$$F^{\otimes n} = F^{\otimes 1} \otimes F^{\otimes (n-1)}$$

$$F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

$$B_N = R_N (I_2 \otimes B_{N/2})$$

$$B_2 = I_2$$

1、假设得到的码序列选择出的四个子信道序号为“4,6,7,8”记作

$$A=\{4,6,7,8\}$$

固定比特序号集合为

$$A^c=\{1,2,3,5\}$$

$$\text{得到 } u_1^8=[0,0,0,1,0,1,1,1]$$

2、构造矩阵，根据公式 $B_N=R_N(I_2 \otimes B_{N/2})$ 计算 B_N

首先 $B_2=I_2$

$$B_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$I_2 \otimes B_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



R_4 由 I_4 变换得来，先排 I_4 的奇数列，再排 I_4 的偶数列：

$$R_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \Leftarrow I_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$B_4 = R_4 (I_2 \otimes B_2) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$I_2 \otimes B_4 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$



R_8 由 I_8 变换得来，先排 I_8 的奇数列，再排 I_8 的偶数列：

$$\begin{aligned}
 R_8 &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \leftarrow I_8 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \\
 B_8 &= R_8(I_2 \otimes B_4) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$



3、求F的n次克罗内克积 $F^{\otimes n} = F \otimes F^{\otimes (n-1)}$

$$F^{\otimes 1} = F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

$$F^{\otimes 3} = F \otimes F^{\otimes 2}$$

$$F^{\otimes 2} = F \otimes F^{\otimes 1}$$

$$F^{\otimes 2} = F \otimes F^{\otimes 1} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \otimes F^{\otimes 1} = \begin{bmatrix} F^{\otimes 1} & 0 \\ F^{\otimes 1} & F^{\otimes 1} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$F^{\otimes 3} = F \otimes F^{\otimes 2} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \otimes F^{\otimes 2} = \begin{bmatrix} F^{\otimes 2} & 0 \\ F^{\otimes 2} & F^{\otimes 2} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$



4、求生成矩阵 G_N $G_N = B_N F^{\otimes n}$

$$G_8 = B_8 F^{\otimes 3} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$



5、生成极化码 $c_1^N = u_1^N G_N$

$$\begin{aligned} c_1^8 = u_1^8 G_8 &= [0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1] \cdot \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \\ &= [0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 1] \end{aligned}$$

速率匹配是把Polar码编码后的比特数量 $N = 2^n$ 通过调整（增加或减少比特数量），适配到对应的物理资源（所承载的比特数量）上。

速率匹配包括两个操作，一是交织，二是按照一定方式选取比特。

速率匹配需要几个参数：

K：（分段后）编码前的信息比特长度。

M：对应资源所承载的比特数。

$R_{\min}$ ：1/8（最小码率）

$N_{\max}$ ：最大母码长度。下行方向为512，上行方向是1024.

$N_{\min}$ ：最小母码长度。固定为32。

N_{dm} ：大于或等于M并且是2的幂的数。

N_{m} ：另一个候选的母码长度。若 $M \leq (9/8) \times N_{\text{dm}}/2$ 且 $K/M < (9/16)$

则 $N_{\text{m}} = N_{\text{dm}}/2$ ；否则， $N_{\text{m}} = N_{\text{dm}}$ 。

速率匹配（选取比特）过程：

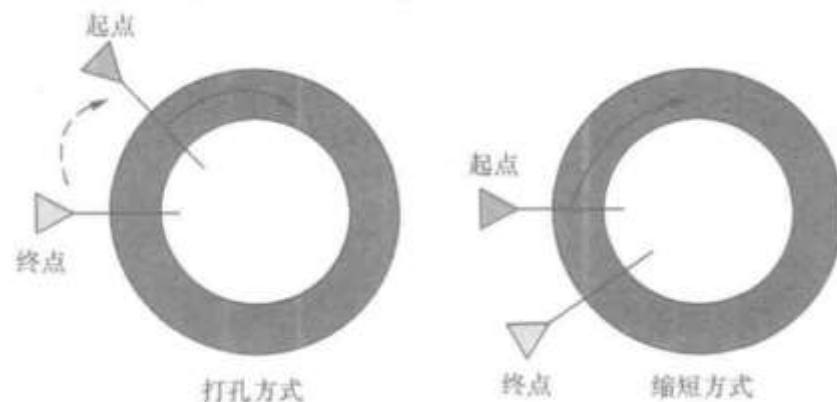
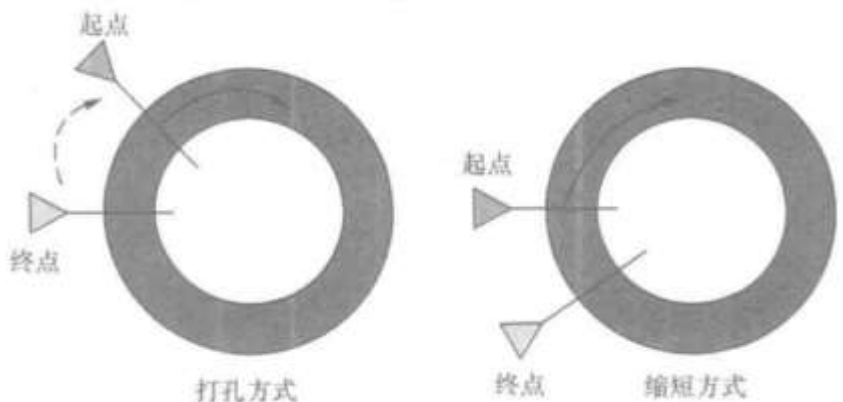
第一步： 将编码之后的比特写入循环缓冲区。

第二步： 若 $M=N$ ，则没有操作。

第三步： 若 $M < N$ ，并且码率 $K/M \leq (7/16)$ ，使用打孔方式。即从第 $N-M+1$ 个比特开始选取，直到最后一个比特。

第四步： 若 $M < N$ ，并且码率 $K/M > (7/16)$ ，使用缩短方式。即从第一个比特开始选取，直到第 M 个比特。

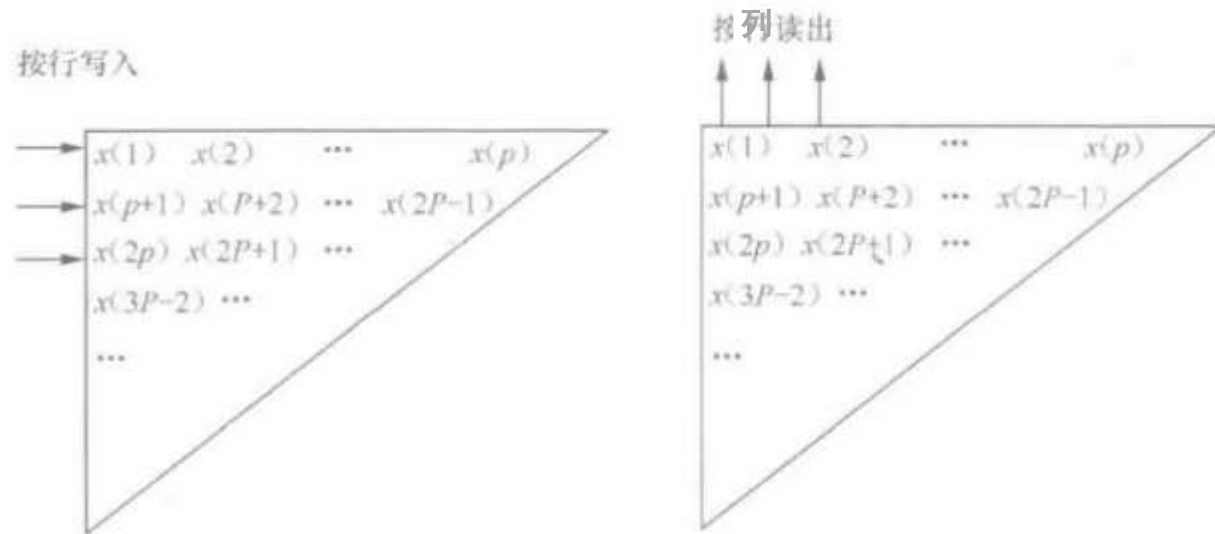
第五步： 若 $M > N$ ，则使用重复方式。除去要取出 N 比特之外，还需要取第一个到第 $\text{mod}[(M-N), N]$ 个比特（对 $M-N$ 取 N 的模）。



交织是指将信息块中的比特顺序打乱，使前后比特之间尽可能减少关联性。有助于对抗突发干扰。本部分讲述后交织（交织规模较大），对时延影响较大。

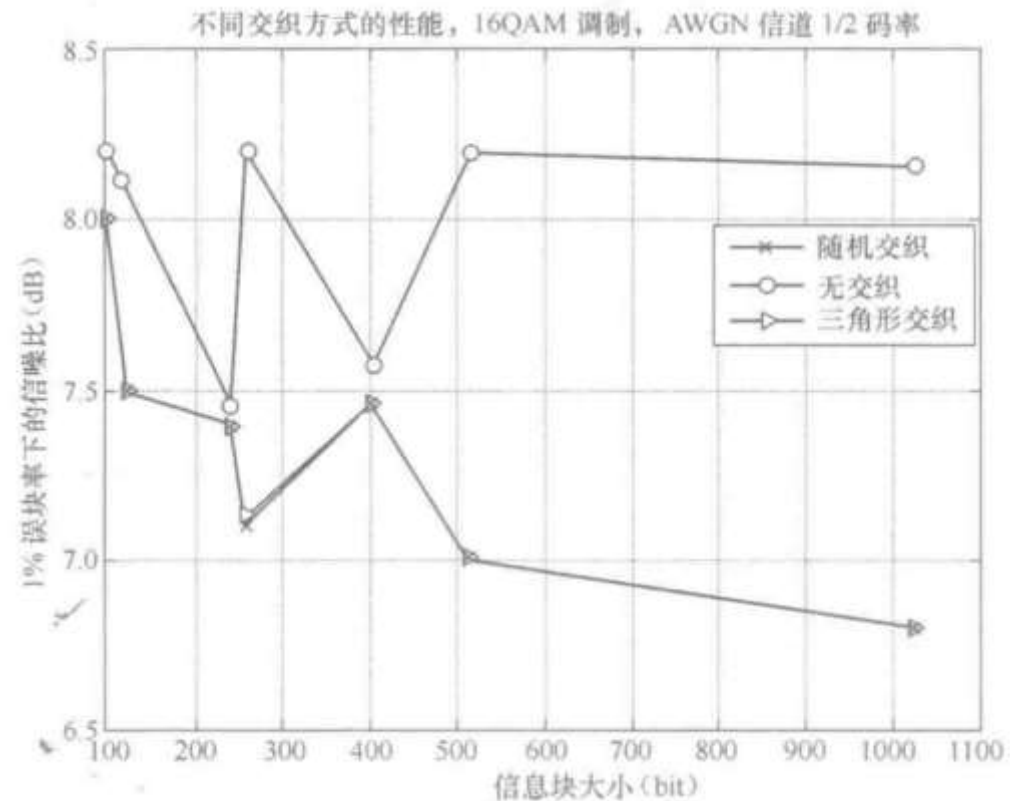
一、等腰直角三角形交织

设直角边长 P （3GPP规定交织器最大8192bit，即 $P \times P/2 \leq 8192$ ， $P \leq 128$ ）， M 为速率匹配后的比特数，边长 P 满足 $P \times (P+1)/2 \geq M$ 。需按行写入，按列读出。如果 $P \times (P+1)/2 > M$ ，需要在最后添加哑元（NULL），读出时碰到NULL则丢弃



等腰三角形交织器重要性质：

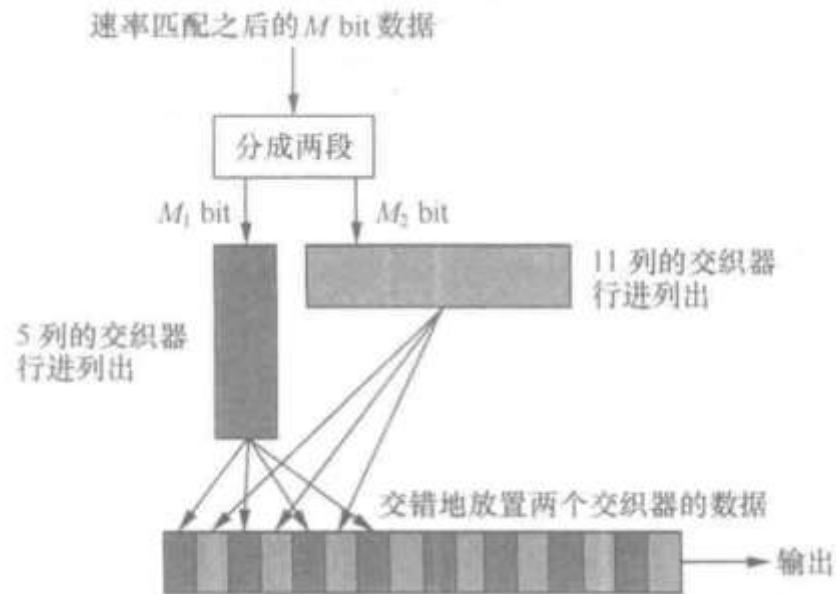
- 交织之前连续排列的数据，在交织之后它们的距离变成 $P, P-1, P-2, P-3 \dots$ 它们之间的距离不再相等。
- 每一列的长度不同，从而使得每列的转置模式都不同。



等腰三角形交织器性能

二、双矩形交织

如图所示双矩形交织（并行矩形交织）中，首先将待交织的 M bit数据分成两个部分： $M_1 = 1 + \text{ceil}(M/2)$ bit和 $M_2 = M - M_1$ bit。其次，把 M_1 bit按行写入深度为5的矩形交织器；把 M_2 bit按行写入深度为11的矩形交织器；必要时，填入NULL。然后，从这两个矩形交织器中按列读出数据。如果有NULL，则丢弃它。最后将这些数据交叠地写入一个 M bit的交织器。



双矩形交织器示意

Polar码解码方式与编码过程一样。即对接收到的信息 y 估计成 c ，然后对 c 进行Polar码编码即可得到原始的原始的发送信息 u 。

由于编码过程： $x = u \cdot G$

等式两边同时右乘 G^{-1} 得到： $u = x \cdot G^{-1}$ （ G^{-1} 是 G 的逆矩阵）

由于 $G^{-1} = (F^{\otimes n})^{-1} = (F^{-1})^{\otimes n} = G$,

所以， $u = x \cdot G^{-1} = x \cdot G$ 。

其中 F 为 $F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ 。

Polar码解码经常使用串行消去（SC）解码方法。

串行消去：

- ① 将比特索引号大的一半（ $N/2$ 个数据）当做随机噪声，根据接收到的数据去解码比索引号小的一半（ $N/2$ 个数据）；
- ② 把解出来的比特索引号小的一半（ $N/2$ 个数据）当做已知比特，根据接收到的数据去解码比特索引号大的一半（ $N/2$ 个数据）。

举例：假设从比特到符号的BPSK映射规则为 $y=2x-1$ （比特“0”映射为“-1”，比特为“1”映射为“+1”），还假设接收到的数据为 y_1, y_2, y_3, y_4 。那么，对这些接收到的数据转换成对数似然比（LLR）

$$x = LLR(y) = \ln \frac{W(y|0)}{W(y|1)}$$

其中， $W(y|0)$ 表示发射端发射的是比特“0”，而接收端收到的是 y 的概率， $W(y|1)$ 表示发射端发射的是比特“1”， $\ln(*)$ 表示自然对数的底 e 为底的对数。

举例： 在现实计算时， $W(y|1)$ 可用下式计算：

$$w(y|1) = \frac{1}{1 + e^{-2y/\delta^2}}$$

其中， δ^2 为噪声方差（可通过对接收到的数据 y 统计得到，一半设置 $\delta^2 = 1$ ）

对于 $W(y|0)$ ，可用 $W(y|0)=1- W(y|1)$ 计算。



N=4解码步骤示意：

第一步：解码器把来自下半部分（ x_3, x_4 ）的信息当做噪声，根据上半部分（ x_1, x_2 ）计算出临时变量 t_1, t_2 。在这里，“当做噪声”的含义是从编码（或数据发射）角度看，有

$$x_1 = t_1 + n_1$$

$$x_2 = t_2 + n_2$$

其中 t_1, t_2 为有用数据， n_1, n_2 为噪声，

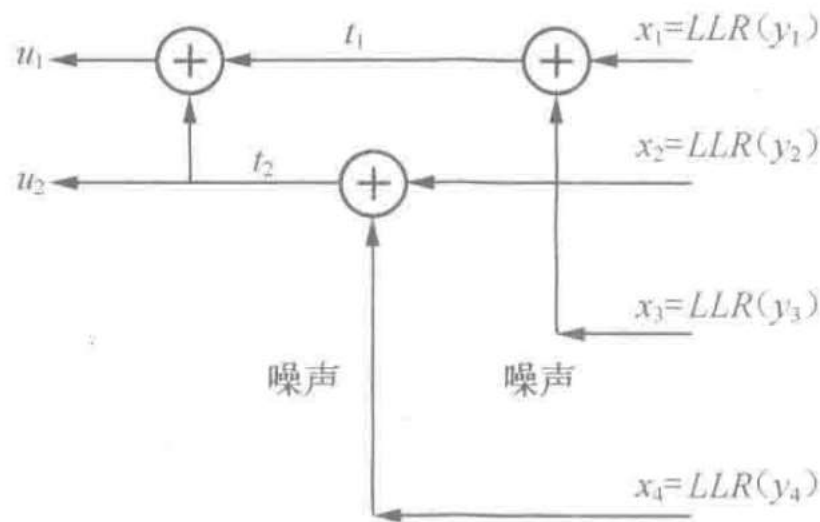
因此在解码时为消除噪声影响需要

$$t_1 = x_1 - n_1$$

$$t_2 = x_2 - n_2$$

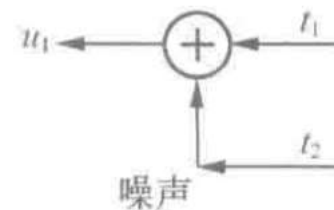
这里的“-”不是简单的减法，二是 f 运算（盒子加，Box-plus），如下。

$$t_1 = x_1 - n_1 = f(x_1, n_1) = x_1 \boxplus n_1 = \ln \frac{1 + e^{x_1 + n_1}}{e^{x_1} + e^{n_1}}$$



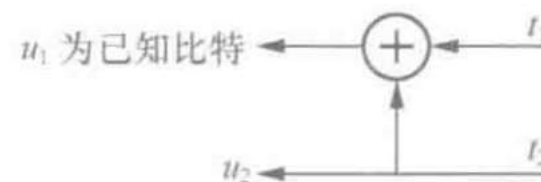
第1步：把下半部分（ x_3 和 x_4 ）看成噪声，去计算出 t_1 和 t_2

第二步：解码器把下半部分（临时变量 t_2 ）看成噪声，求解出 u_1 。



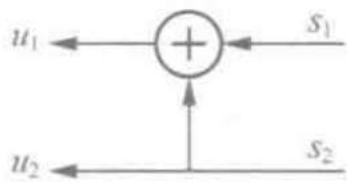
第2步：把下半部分（ t_2 ）看成噪声，去解出 u_1

第三步：把上半部分（ u_1 ）当做已知比特，去解出 u_2 。此时应根据 u_1 的取值（0,1）来对临时变量进行 t_1, t_2 的操作，然后再译出 u_2 。



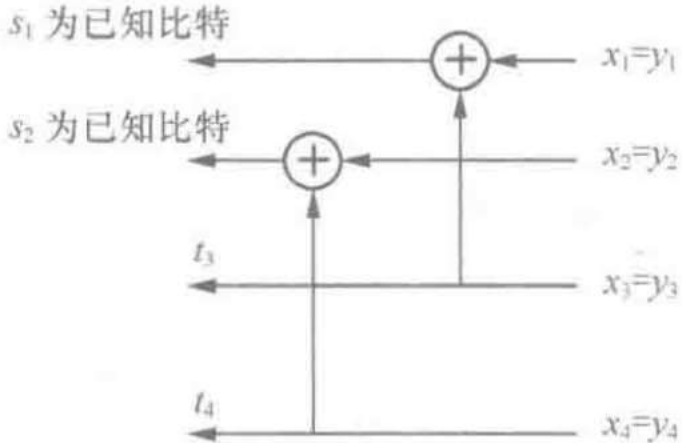
第3步：把上半部分（ u_1 ）当作已知比特，去解出 u_2

第四步：计算出另一组临时变量 $s_1 = u_1 + u_2$ （这里指模2加“ $\oplus$ ”）和 $s_2 = u_2$



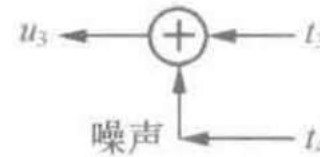
第 4 步：计算出 $s_1 = u_1 + u_2$ 和 $s_2 = u_2$

第五步：把上半部分（临时变量 s_1 和 s_2 ）当做已知比特，去计算又一组临时变量 t_3, t_4



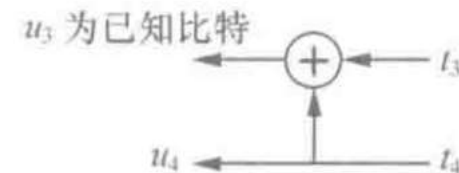
第 5 步：把上半部分（ s_1 和 s_2 ）
当作已知比特，去计算出 t_3 和 t_4

第六步：与第二步类似，把下半部分（临时变量 t_2 ）看成噪声，求解出 u_3 。



第6步：把下半部分（ t_4 ）
看成噪声，去解出 u_3

第七步：与第三步类似，把上半部分（ u_3 ）当做已知比特，去解出 u_4 。



第7步：把上半部分（ u_3 ）
当作已知比特，去解出 u_4

例3：假设接收到数据为 $y_1=-1.61$ 、 $y_2=-0.06$ 、 $y_3=-1.89$ 、 $y_4=1.69$ ，因为 $x=LLR(y)$ ，所以 $x_1=3.22$ 、 $x_2=0.12$ 、 $x_3=3.78$ 、 $x_4=-3.38$ 。计算解码结果。

第一步：将 x_3, x_4 当做噪声，根据上半部分 x_1, x_2 计算出临时变量 t_1, t_2 。

$$t_1 = f(x_1, x_3) = x_1 \boxplus x_3 = 2.77$$

$$t_2 = f(x_2, x_4) = x_2 \boxplus x_4 = -0.11$$

第二步：解码器把下半部分（临时变量 t_2 ）看成噪声，求解出 u_1 。那么，

$$z_1 = f(t_1, t_2) = t_1 \oplus t_2 = -0.09$$

对 z_1 进行判决（若是冻结比特，则无需判决，直接输出冻结比特值），
得到 $u_1=1$ （这一比特解码错误；从仿真程序看，SNR=0dB，下同；考虑码长
N=4不够大，极化现象不明显，且码率R=1，有错误正常）

$$\text{判别条件: } u_i = \begin{cases} 0, & \text{如果 } LLR_i \geq 0 \\ 1, & \text{其他} \end{cases}$$

第三步：把上半部分 (u_1) 当做已知比特，去解出 u_2 。此时应根据 u_1 的取值 (0,1) 来对临时变量进行 t_1, t_2 的操作，然后再译出 u_2 。

根据编码规则，如果 $u_1=0$ ，则编码之后有 $t_1 = t_2$ ，那么对 u_2 进行解码时， $z_2 = t_1 + t_2$ ，对 z_2 进行判决得到 u_2 ；

若 $u_1=1$ ，则在编码之后有 $t_1 = 1 - t_2$ （经BPSK后 $t_1 + t_2 = 0$ ），那么对 u_2 解码时，有 $z_2 = t_2 - t_1$ ，对 z_2 进行判决，得到 $u_2=1$ （解码正确）

第四步：计算出另一组临时变量 $s_1 = u_1 \oplus u_2$ 和 $s_2 = u_2$ 。根据上面第2~3步

结果得到 $s_1=0$, $s_2=1$ 。

第五步：把上半部分（临时变量 s_1 和 s_2 ）当做已知比特，去计算又一组临时

变 量 t_3, t_4 。类似于第三步，有

$$t_3 = x_3 + (1 - 2 \times s_1) \cdot x_1 = 3.78 + (1 - 2 \times 0) \times 3.22 = 7.00$$

$$t_4 = x_4 + (1 - 2 \times s_2) \cdot x_2 = -3.38 + (1 - 2 \times 1) \times 0.12 = -3.50$$

第六步：与第二步类似，把下半部分（临时变量 t_2 ）看成噪声，求解出 u_3 。

$$z_3 = f(t_3, t_4) = t_3 \oplus t_4 = -3.47$$

对 z_3 进行判决，得到 $u_3=1$ （解码错误）

第七步：与第三步类似，把上半部分（ u_3 ）当做已知比特，去解出 u_4 。

$$z_4 = t_4 + (1 - 2 \times u_3) \times t_3 = -3.50 + (1 - 2 \times 1) \times 7.00 = -10.50$$

对 z_4 进行判决，得到 $u_4=1$ （解码正确）

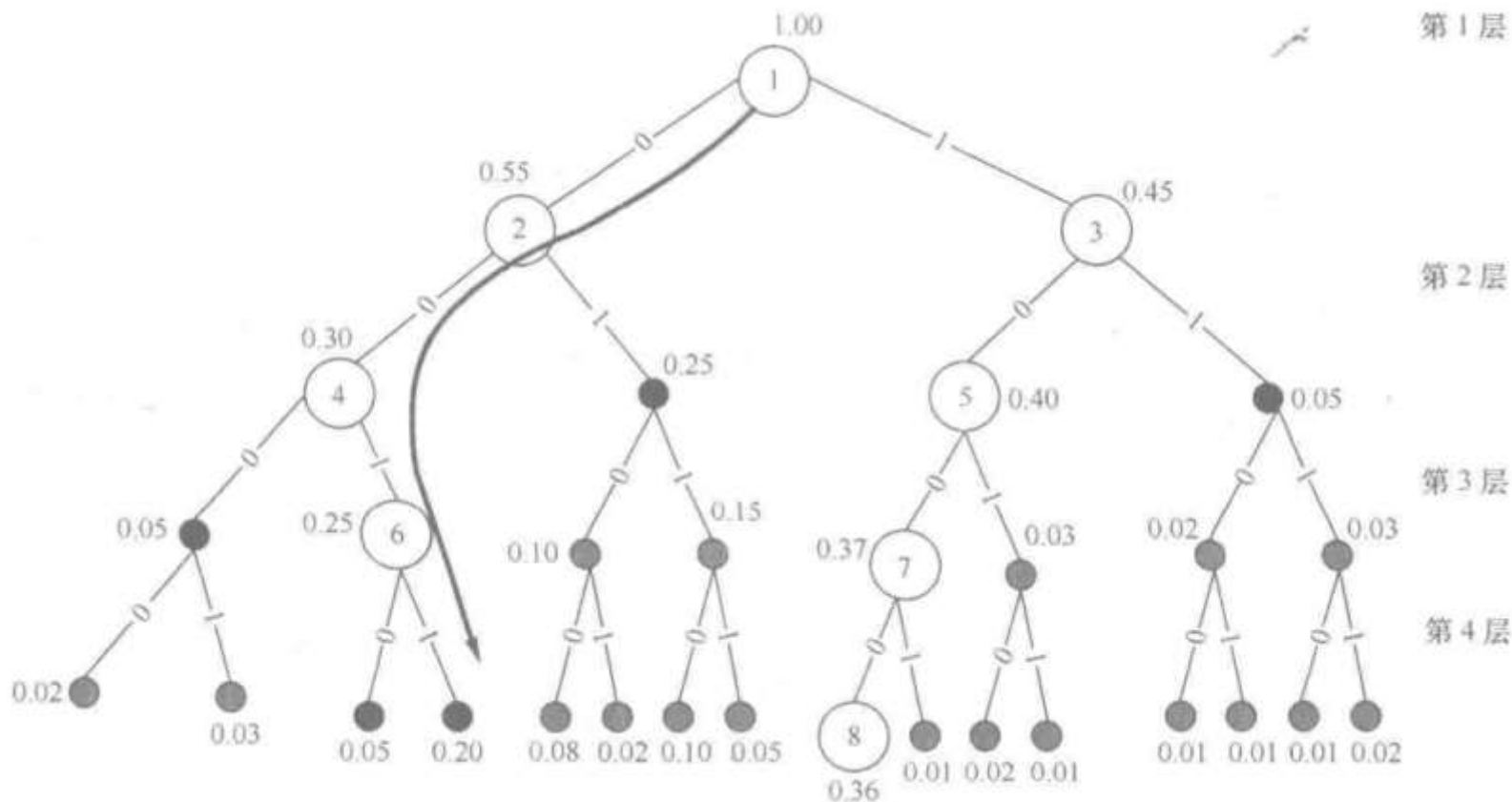
小结：

上述解码方法是串行进行的（先解 u_1 ，然后解 u_2 ，再解 u_3 ，最后解 u_4 ）并且在解码后面的比特（如 u_2 ）时，充分考虑了前面比特（ u_1 ）的影响，从而逐步消除前面比特的影响，故称之为串行消除算法。

当母码长度更大的时候，需要执行多次上面这些步骤。



普通SC算法解码方式主要通过后验概率进行判决，总是通过最大后验概率的路径进行分析，但通过SC算法，有时可能会走错。（码树进行表示）



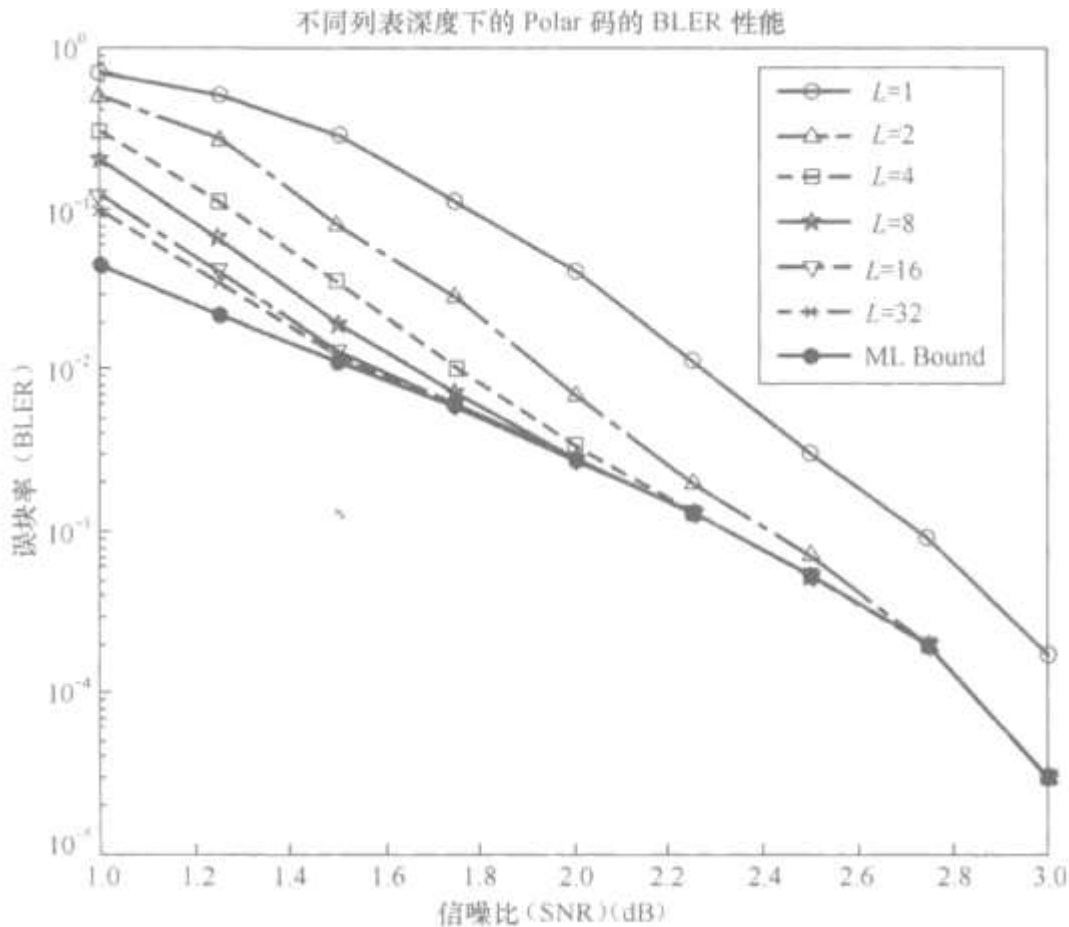
SC算法码树表示

第二种方式:SC-L算法

SC-L算法是在SC算法的基础上，通过保存L条具有最大后验概率的路径来进行下一层的解码，算法在最后选取一条具有最大后验概率的路径。即相对SC算法，SC-L算法增加了路径保存和回退功能。

当列表深度 $L=1$ 时，SC-L算法等同于SC算法；

当列表 $L=\min(2^N, 2^{\text{Length}(A)})$ 时，SC-L算法就变成了ML算法。 $\text{Length}(A)$ 为信息集的长度。当列表长度 $L=32$ 时，SC-L算法已经与ML非常接近了。



以母码长度 $N=4$ 为例，用码树表示，设列表深度 $L=2$ 。

第一层，SC-L算法按照最大APP找到左边路径 $\{0, x, x, x\}$ 和右边路径 $\{1, x, x, x\}$ 。找到这样两条路径，并保存到 $L=2$ 条列表中。

第二层，SC-L算法按照最大APP找到左边路径 $\{0, 0, x, x\}$ 和右边路径 $\{1, 0, x, x\}$ 。

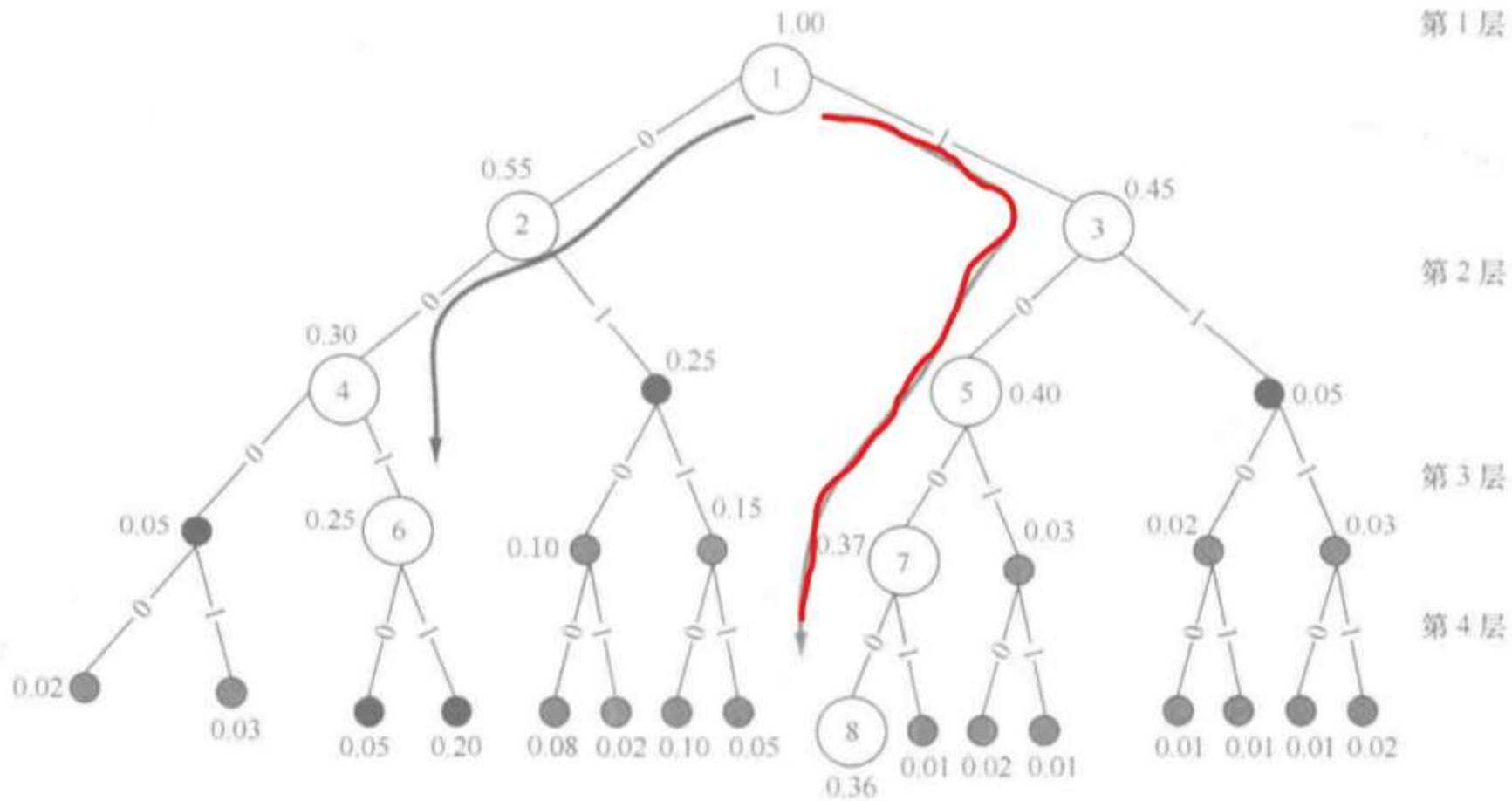
第三层，SC-L算法按照最大APP找到左边路径 $\{0, 0, 1, x\}$ 和右边路径 $\{1, 0, 0, x\}$ 。

第四层，SC-L算法按照最大APP找到左边路径 $\{0, 0, 1, 1\}$ 和右边路径 $\{1, 0, 0, 0\}$ 。

因为右边路径的APP超过左边路径，因此SC-L算法选择右面路径输出解码比特 $\{1, 0, 0, 0\}$ ，从而完成解码。



解码算法



复杂度是指一种编码方案实现需要多大的计算量、存储量，其中包含编码的复杂度（大）与解码的复杂度（小），复杂度与吞吐量存在一定的平衡点。

SC算法: 编码复杂度和解码复杂度都是 $O[N \cdot \log_2(N)]$

因为长度为 N 的 Polar 码，有 $\log_2(N)$ 个层分支，在编码时需要进行 $(\log_2 N) \cdot N/2$ 次模 2 加操作，即编码复杂度 $O[N \cdot \log_2(N)]$ ；

解码时，需要 $N \cdot \log_2(N)/2$ 次校验节点操作和 $N \cdot \log_2(N)/2$ 次变量节点操作，复杂度同样为 $O[N \cdot \log_2(N)]$

SC-L算法: 解码复杂度是 $O[L \cdot N \cdot \log_2(N)]$ L 为列表深度。

SC算法: 存储复杂度为 $O[N \cdot \log(N)]$

对于长度为 N 的 Polar 码，有 $\log_2(N)$ 个层分支，每层存储 N 个似然比值，为找出原始发射路径，需存储 $N \times \log_2(N)$ 个似然比值。

SC-L算法: 存储复杂度为 $O[L \cdot N \cdot \log(N)]$

因为每一个列表都要有一份后验概率值。

总结来说，Polar 码对于存储量的要求较低

Polar 码解码算法复杂度与信噪比基本无关

2016年11月14日至18日期间，国际无线标准化机构（3GPP）无线物理层(RAN1) 第87会议在美国Reno召开，会议上，经过激烈的竞争，华为等中国企业主推的极化码（Polar Code）击败美国主推的LDPC码和法国主推的Turbo码，成为5G eMBB场景在短码上的控制信道编码方案。

自此，在5G eMBB场景上，Polar码和 LDPC 码二分天下，

信令信道编码方案

数据信道编码方案

- Polar码历史性地与LDPC码一起走进蜂窝移动通信系统。
- Polar码入选5G标准意味着中国通讯企业第一次迈入世界顶尖领域。

为什么目前我国5G基站的信道编码方式选用极化码？

极化码的优势是计算量小，用小规模的芯片就可以实现，比较适应于5G的小基站，而且采用这种编码方法的硬件商业化后设备成本低，因此也极具竞争力。

思政学习

虽然极化码的原始方案不是由华为首提的，但是从商业与应用角度来讲，这是中国通信技术的一次巨大成功。中国公司首次进入基础通信框架协议领域，最重要的意义在于加大了中国企业在全球通信领域的话语权。

华为主导推动的极化码被3GPP采纳，是中国在5G通信技术研究和标准化上的重要进展。期待中国在接下来的技术角逐中能够有更多的表现和收获。

思政学习

今天，华为不光研发出了中国自己的5G标准，更研发出了携有5G网络基带的国产芯片！作为华为，乃至中国的第一代智能芯片，不仅彻底打破了国外大企业的垄断，更是将中国的移动网络安全，重新掌控到了自己的手里。

华为公司的科研人员以严谨科学的态度和自立自强的勇气，践行“严谨求实、团结协作、拼搏奉献、勇攀高峰”的精神，为科技创新树立了典范。

这一次，中国要用自己的实际行动郑告整个世界：中国，要引领新的时代！

发展方向：

- 1、不同于Arikan核的极化核，可采用高维的极化核加快极化速度（可能导致多个极化值）
- 2、空时极化编码，把各个天线发射的码流用Polar码方式进行编码。
- 3、增加外码，如RS码，有助于进一步提升系统性能。
- 4、采取低复杂度并行解码。

思考？

- 目前极化码如何在短码情况下提高解码性能？



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第九章 纠删码

李丽香，彭海朋

北京邮电大学网络空间安全学院

网络与交换技术国家重点实验室



Wikipedia: In coding theory, an erasure code is **forward error correction** (FEC) code under the assumption of bit erasures (rather than bit errors), which transforms a message of k symbols into a longer message (code word) with n symbols such that the original message can be recovered from a subset of the n symbols.



在编码理论中，有一种前向纠错技术(FEC)也称为纠删码。

这种技术可以将原始数据中丢失的 k 字节数据从 n 个含编码字节的信息中恢复。

01

在香农定理的指导下，信道编码理论和技术逐步发展成熟。

02

纠删码(Erasure Code)原来是**无线通信中有噪信道编码的一种**。

03

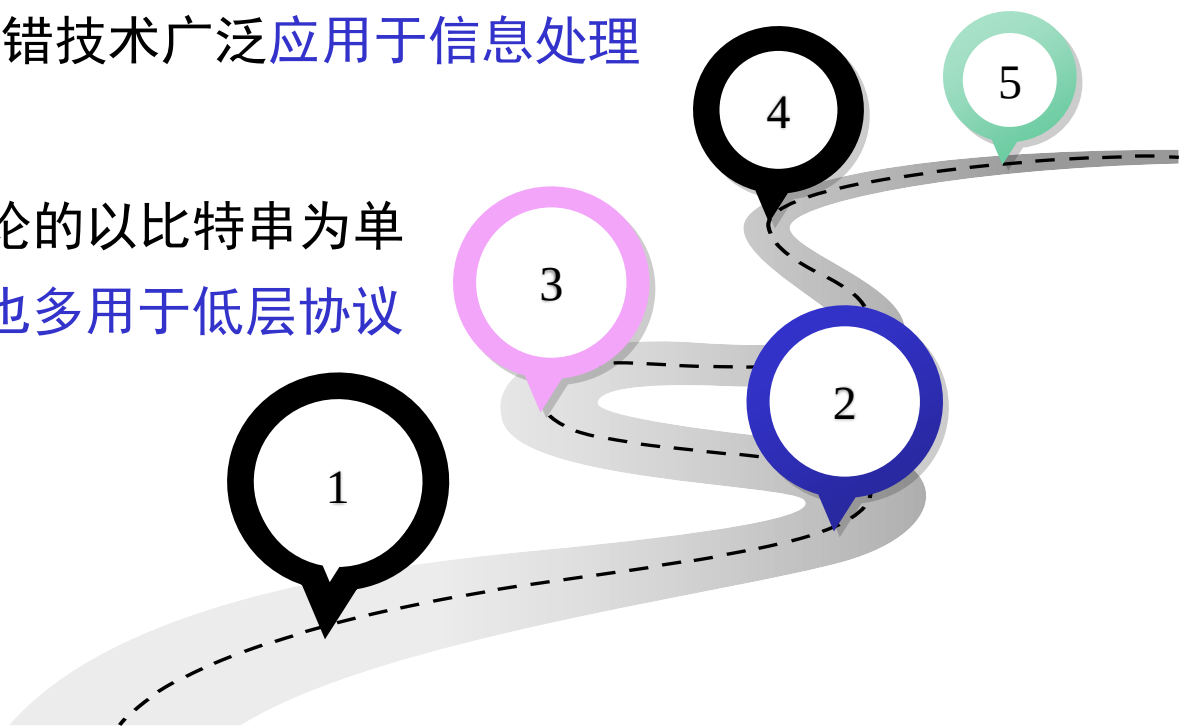
Karnin提出的密钥共享(Secret Sharing)以及M.O.Rabin提出的IDA (Information Dispersal Algorithm)算法，将其引入到计算机应用中。

04

近年来，前向纠错技术广泛**应用于信息处理的各个领域**。

05

各种纠错码所讨论的以比特串为单位的**错误检测和纠错也多用于低层协议(如链路层)**。



1

数据的存储呈现出集中化(以分布式存储系统为基础的云存储系统)和移动化(互联网移动终端)的趋势，**数据可靠性**愈发引起重视。

2

集群所承载的数据量大大上升，但存储介质本身的可靠性进步却很小，这要求必须以**更加经济有效的方式来保障数据安全**。

3

副本技术通过**增加冗余数据的方式**来保证数据在发生部分丢失时，原始数据不发生丢失。但相较于副本，纠删码能以**低得多的存储空间代价获得相似的可靠性**。

要想知道 x_1 、 x_2 、 x_3 的值，只需要任意3个方程式即可解出。

$$x_1 = 1 \quad (1)$$

$$x_2 = 2 \quad (2)$$

$$x_3 = 3 \quad (3)$$

$$x_1 + x_2 + x_3 = 6 \quad (4)$$

$$x_1 + 2x_2 + 4x_3 = 17 \quad (5)$$

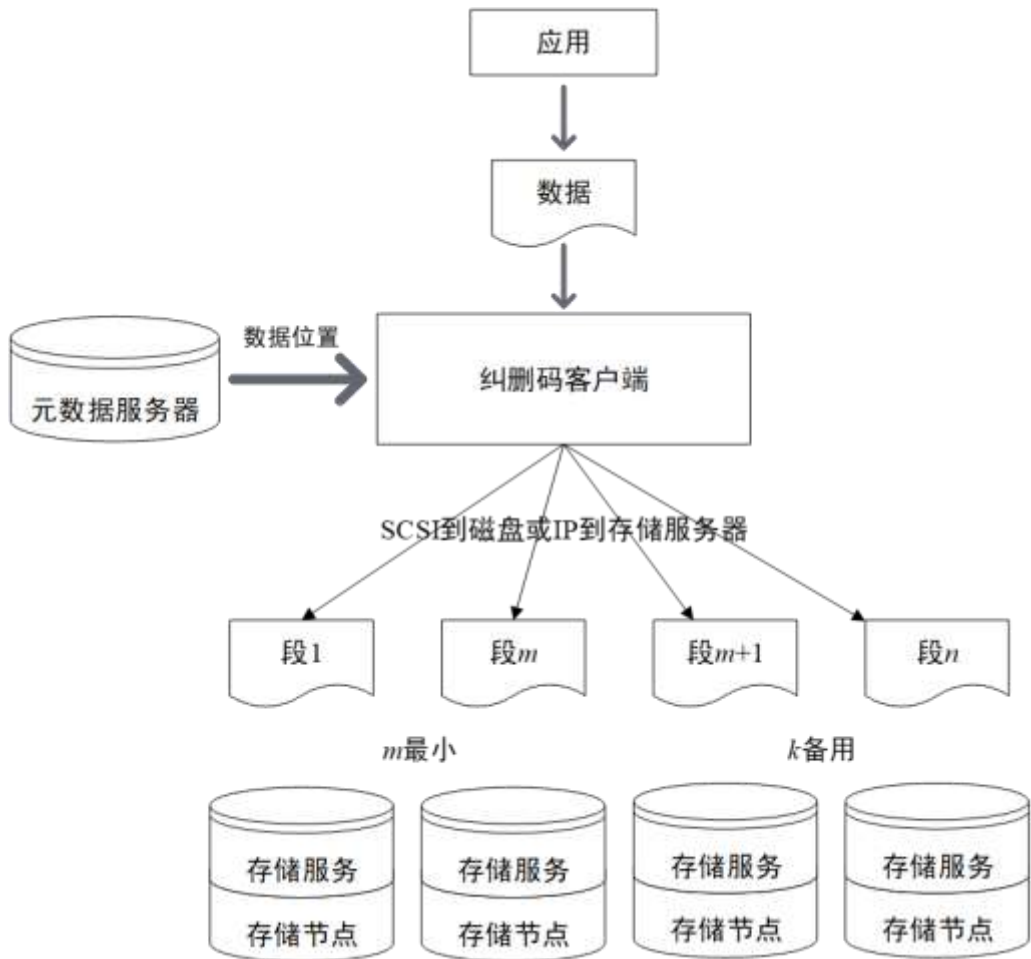
$$x_1 + 3x_2 + 9x_3 = 34 \quad (6)$$

副本策略：若方程(1)(2)(3)丢失任何一个方程数据都能恢复，则需要把(1)(2)(3)三个方程都存储两份，也就是存储6个方程。

纠删码策略：把方程(1)(2)(3)看做是分布式系统的数据，(4)(5)(6)看做是code，只要一个code，即使丢了(1)(2)(3)中的任何一个方程，数据都可以恢复，故只需要存储4个方程。

策略存储效率对比：纠删码策略存储4个方程，副本策略存储6个方程，存储效率是4/6，可以节省1/3的空间。实际根据code的多少，存储效率会不一样。

纠删码是一组数据冗余和恢复算法的统称，是一种数据保护方法。

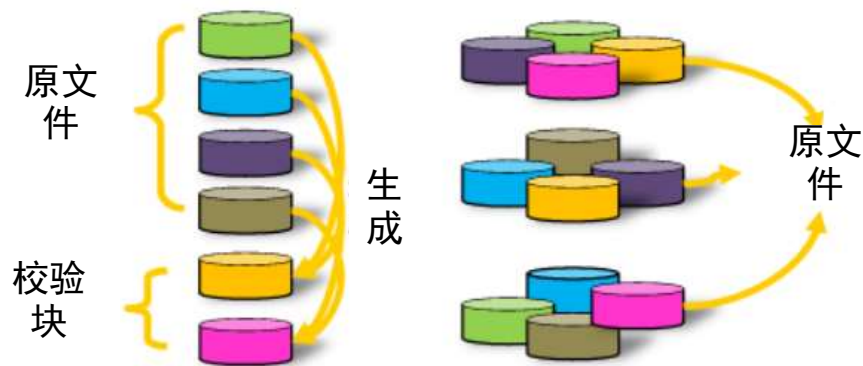


纠删码的存储思想

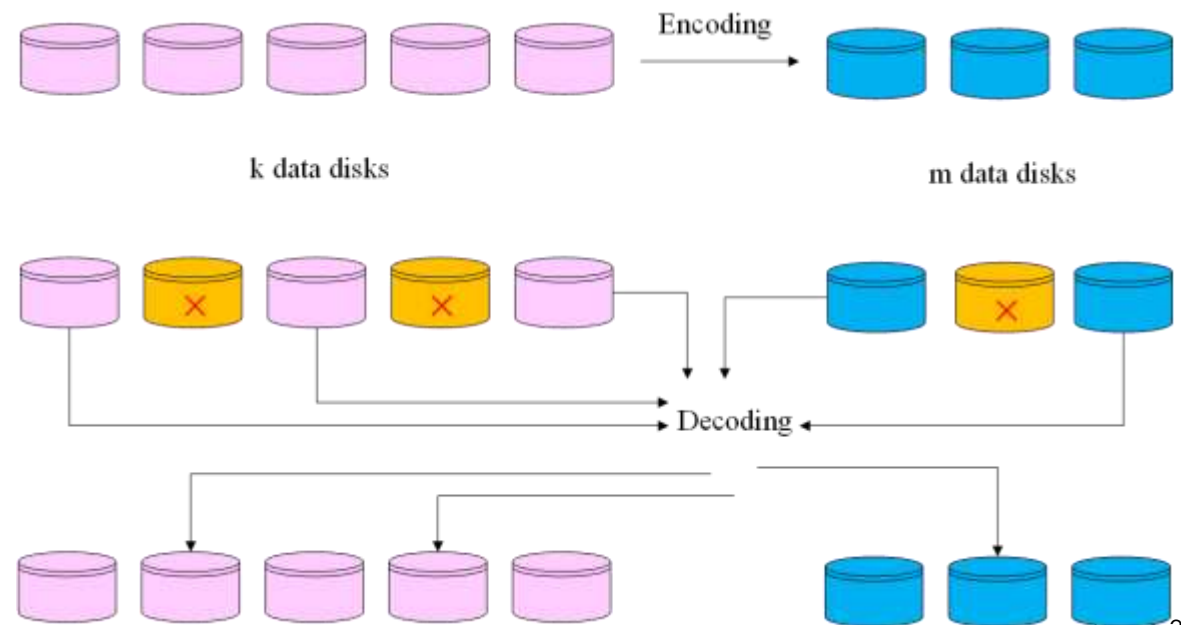
纠删码技术将数据**分割成片段**，把冗余数据块扩展、编码，并将其**存储在不同的位置**，比如磁盘、存储节点或者其它地理位置。

纠删码技术**具有良好的容错性和安全性**，主要应用于网络传输中丢包错误的解决。

网络传输方面：纠删码的基本思想是“发送方将 n 个源数据包编码，得到 m 个校验数据包，将这 $n+m$ 个数据包在网络上进行传输。接受方只要接收到不少于 n 个编码包，就可以将源数据恢复。一般来说，这个传输过程允许最多 m 个数据包的丢失。”



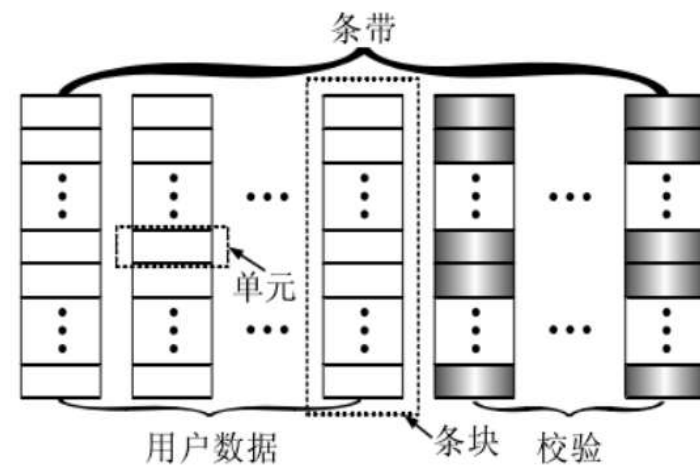
数据存储方面：纠删码的基本思想是“当冗余级别为 $n+m$ 时，将这些数据块分别存放在 $n+m$ 个硬盘上，这样就能容忍 m 个(假设初始数据有 n 个)硬盘发生故障。当不超过 m 个硬盘发生故障时，只需任意选取 n 个正常的数据块就能计算得到所有的原始数据。”



单元(Element)是用户数据或校验的一个基本单位，可以是一个比特(Bit)、一个字节(Byte)、一个扇区(Sector)或一个更大的磁盘数据块。它是纠删码的构造块(Building Block)。在编码理论中，它对应于码符号(Code Symbol)中的一个比特。

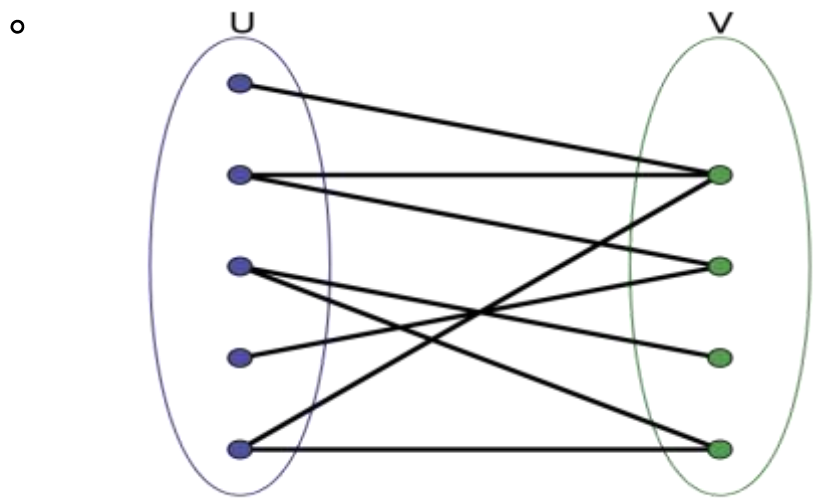
条带(Stripe)是一个由用户数据单元和校验单元组成的具有独立编码关系的集合。在编码理论中，它对应于一个码字(Code Word)。这里，条带长度(Stripe Length)定义为一个条带所跨越的磁盘个数，记为 n 。

条块(Strip)是一个条带中存储在同一个磁盘上的所有单元组成的集合。在编码理论中，它对应于码字中的一个码符号。这里，条块宽度(Strip Width)定义为一个条块中包含的单元个数，记为 w 。



用户数据分布图

二部图又称为二分图，是图论中的一种特殊模型。设 $G=(V,E)$ 是一个无向图，如果顶点 V 可以分割为两个互不相交的子集 (U,V) ，并且图中每条边 (i,j) 所关联的两个顶点 i 和 j 分别属于这两个不同的顶点集($i \in U, j \in V$)，则称图 G 是一个二部图。



二部图示例

范德蒙矩阵是法国数学家范德蒙提出的一种各列为几何级数的矩阵。

该矩阵第 i 行、第 j 列可以表示为 g_j^{i-1} 。

$$G = \begin{bmatrix} g_0^1 & g_1^1 & g_2^1 & \cdots & g_{n-1}^1 \\ g_0^2 & g_1^2 & g_2^2 & \cdots & g_{n-1}^2 \\ g_0^3 & g_1^3 & g_2^3 & \cdots & g_{n-1}^3 \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ g_0^m & g_1^m & g_2^m & \cdots & g_{n-1}^m \end{bmatrix} \quad \begin{bmatrix} \frac{1}{x_1+y_1} & \frac{1}{x_1+y_2} & \cdots & \frac{1}{x_1+y_n} \\ \frac{1}{x_2+y_1} & \frac{1}{x_2+y_2} & \cdots & \frac{1}{x_2+y_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{x_{m-1}+y_1} & \frac{1}{x_{m-1}+y_2} & \cdots & \frac{1}{x_{m-1}+y_n} \\ \frac{1}{x_m+y_1} & \frac{1}{x_m+y_2} & \cdots & \frac{1}{x_m+y_n} \end{bmatrix}$$

柯西矩阵中 $\{x_1, x_2, \dots, x_m\}$ 和 $\{y_1, y_2, \dots, y_n\}$ 为同一个有限域的两个不同子集。它们必须满足：

- (1)不同子集中的任意两个元素相加不能为0。
- (2)这两个子集中的每个元素都不相同。

这两个条件都满足才能构成柯西矩阵。

伽罗华域，用 $GF(2^w)$ 来表示，代表含有 2^w 个元素的有限域。伽罗华域中的四则运算(即基于多项式的运算)，多项式一般为 $f(x) = x^6 + x^4 + x^2 + x + 1$ 。

(1)多项式加减法

将两个多项式中相同阶数的项系数相加或相减。例如 $(x^2 + x) + (x + 1) = x^2 + 2x + 1$

(2)多项式乘法

将其中一个多项式的各项分别与另一个多项式的各项相乘，然后把相同指数的项的系数相加。例如 $(x^2 + x)(x + 1) = x^2(x + 1) + x(x + 1) = x^3 + x^2 + x^2 + x$

(3)多项式除法

使用长除法。例如计算 x^3-12x^2-42 ，除以 $x-3$ 。使用长除法计算，商 $x^2-9x-27$ ，余数-123。

$$\begin{array}{r} x^2 - 9x - 27 \\ x - 3 \overline{) x^3 - 12x^2 + 0x - 42} \\ \underline{x^3 - 3x^2} \\ -9x^2 + 0x \\ \underline{-9x^2 + 27x} \\ -27x - 42 \\ \underline{-27x + 81} \\ -123 \end{array}$$

(4) $GF(2^w)$ 上的多项式运算

对于 $GF(2^w)$ 上的多项式计算，多项式系数只能取0或1。(如果是 $GF(3^w)$ ，那么系数可以取0、1、2)

$GF(2^w)$ 的多项式加法中，合并阶数相同的同类项时，由于 $0+0=0, 1+1=0, 0+1=1+0=1$ ，因此系数不是进行加法操作，而是进行异或操作。

$GF(2^w)$ 的多项式减法等于加法，例如 $x^4 - x^4$ 就等于 $x^4 + x^4$ 。

编码方式：

将原数据重复 k 次后，就得到 k -重复码。例如，将原数据 x 重复3次，拼接得到数据 $\{x, x, x\}$ ，即为原数据的3-重复码。发送方传输的就是 k -重复码。

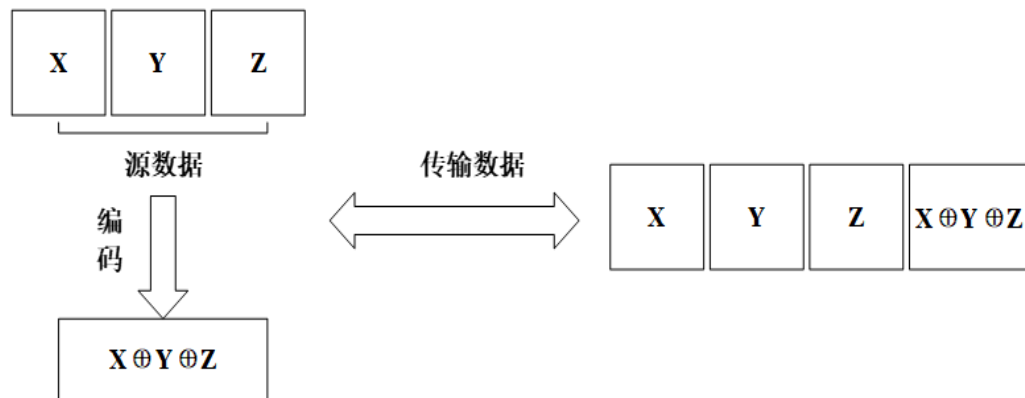
解码方式：

将接收到的数据分为 k 段， k 为原数据的重复次数。每一段即是一个原数据，这样就完成了数据恢复。将这 k 段数据比较。可保证传输数据的可靠性。

编码方式：

奇偶校验码是一种通过增加冗余位，使得码字中“1”的个数恒为奇数或者偶数的编码方法。

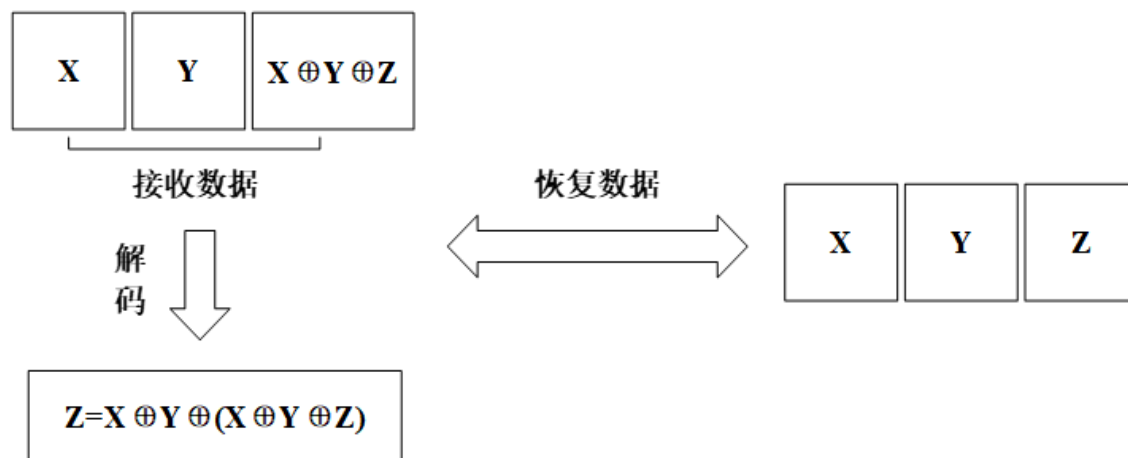
将原数据分为 k 个等长的部分，将这 k 个部分进行异或操作，在数据最后加上异或操作的结果，作为冗余段，然后将原数据和冗余段一起传输。



奇偶校验码编码示例

解码方式：

冗余检测方式，接收方根据接收到的数据和冗余恢复出丢失的数据。也可以对接收到的数据进行异或操作，与数据段尾部的冗余比较，以保证数据传输的可靠性。



奇偶校验码解码示例

重复码可容忍擦除率很高，但编码率低，效率低下，造成浪费。

奇偶校验码的编码率高，但可容忍擦除数少，容错性差。

二者的具体区别如表所示。其中 n 为原数据个数， m 为冗余数。

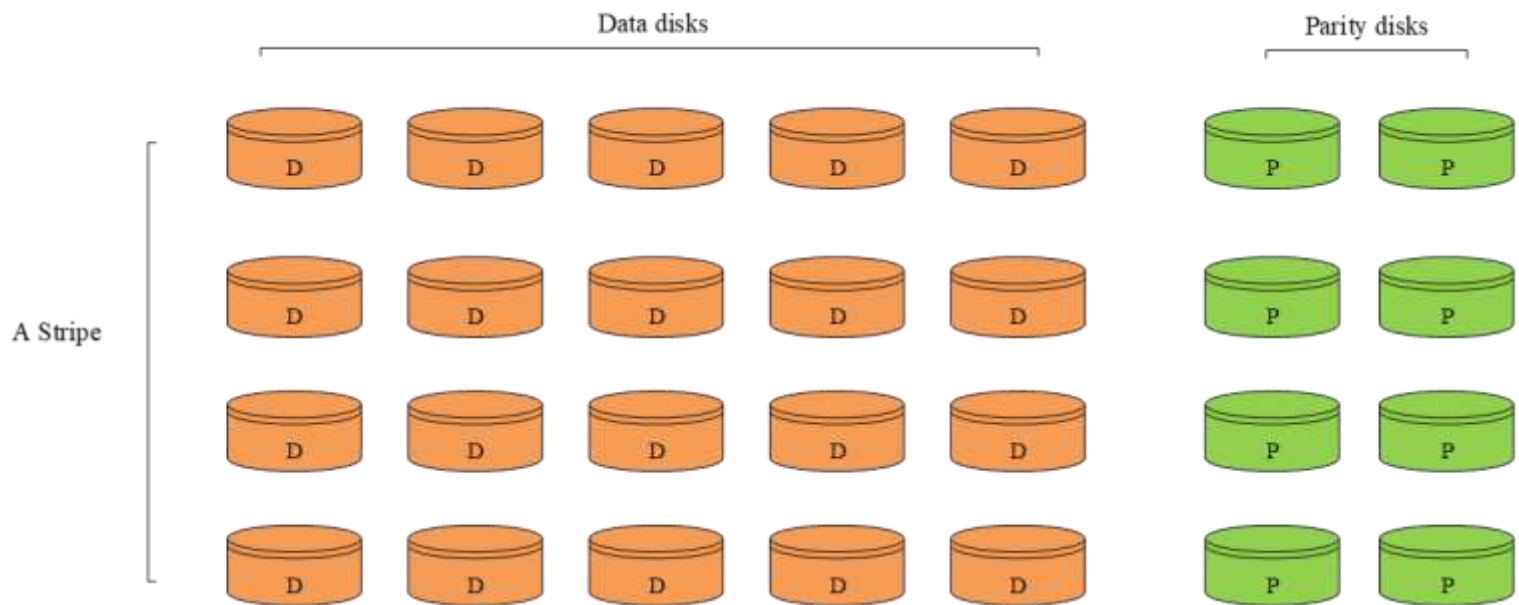
	重复码	奇偶校验码
可容忍擦除数	最多 $(k-1)$ 倍于原数据包	只能容忍1个数据包
编码率	低，只有 $1/n$	高达 $n/(n+m)$

目前，纠删码技术在分布式存储系统中的应用主要有四类。



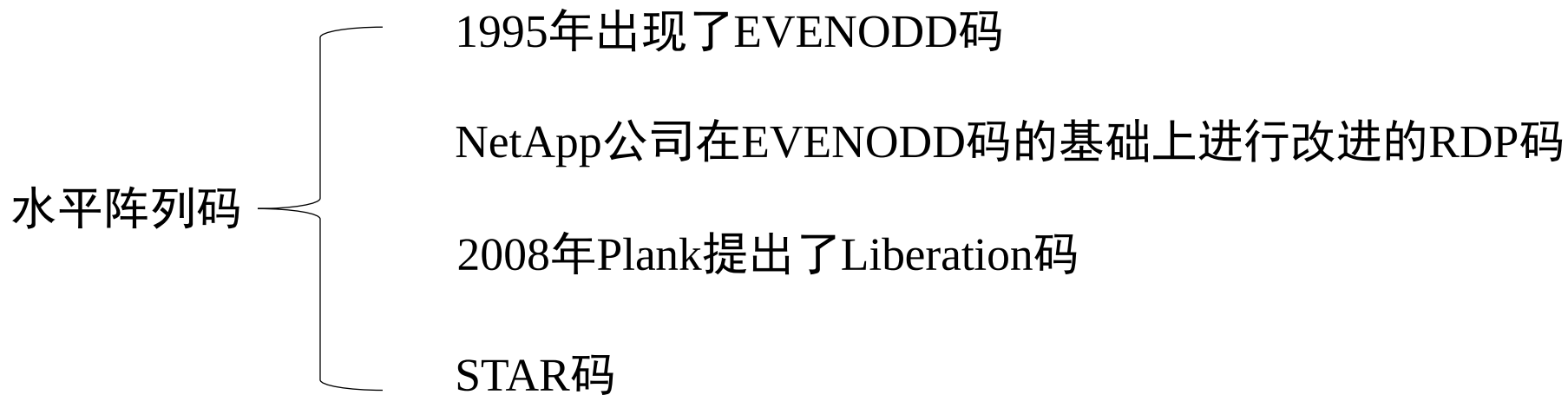
阵列码(Array Code), 即奇偶校验阵列码(Parity Array Code)的简称。它将一个条带中的用户数据单元和校验单元排列成一个二维或多维阵列, 并基于XOR运算对条带内的单元进行编码。阵列码的提出主要是为了满足高容错磁盘阵列设计的需要。

阵列码分为水平阵列码、垂直阵列码和水平垂直阵列码。

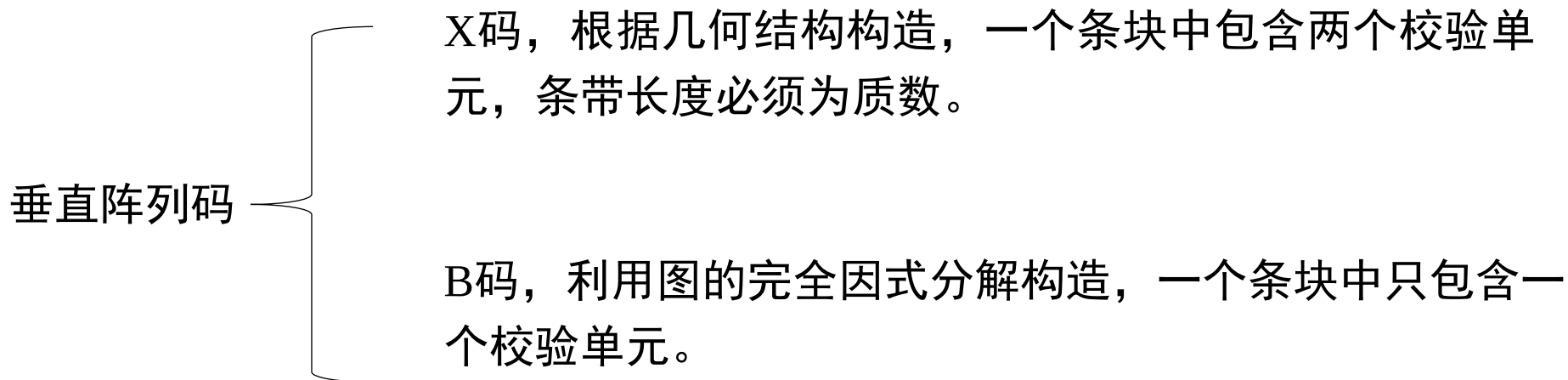


阵列码布局

水平阵列码的一个条带中的用户数据单元和校验单元分别存放在不同的条块中。



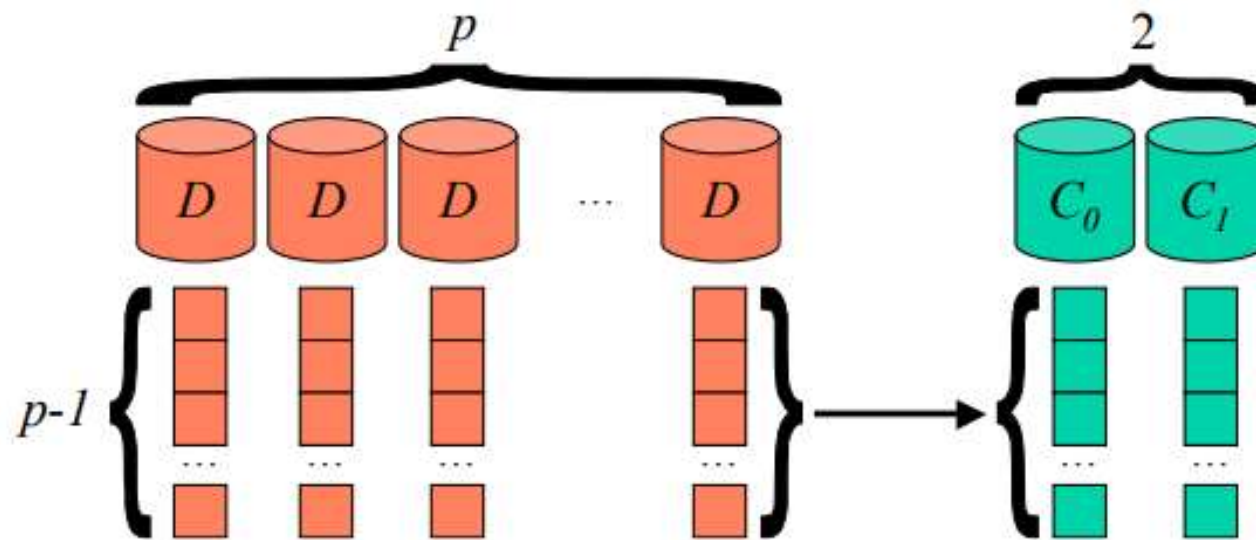
垂直阵列码的一个条带中的每个条块既包括用户数据单元，又包括校验单元。垂直阵列码的出现是为了使更新复杂度达到最低。



水平垂直阵列码的一个条带中一些条块只包括校验单元，另一些条块同时包括用户数据单元和校验单元。

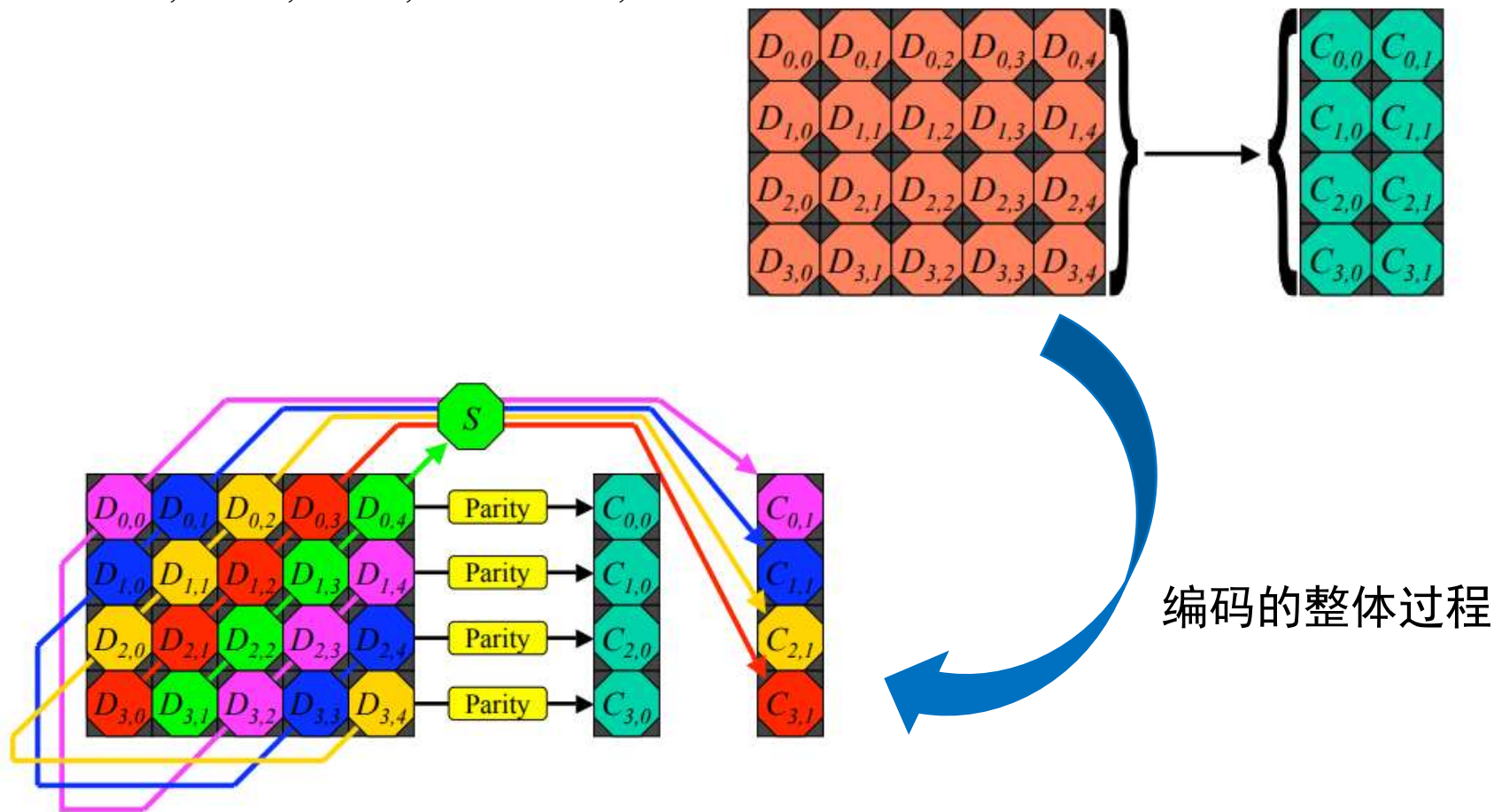


在阵列码中，EVENODD码是首个可以容忍两列丢失的编码方法，被公认为阵列码的始祖。在EVENODD码中，一共有 p 列数据列和2列冗余校验列，每一列有 $p-1$ 个信息位，因此其码字为 $(p-1)*(p+2)$ 的二维阵列，其中 p 为大于2的素数。



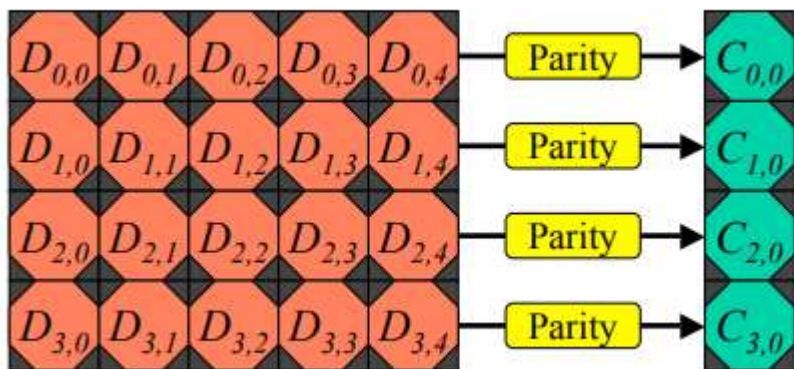
EVENODD码布局

$p=5$ 时，EVENODD的码字如下图所示，EVENODD编码的目的是计算出 $C_{0,0}$ 、 $C_{1,0}$ 、 $C_{2,0}$ 、...、 $C_{3,1}$ 。



(1) 计算第一列校验信息即 $C_{0,0}$ 、 $C_{1,0}$ 、 $C_{2,0}$ 、 $C_{3,0}$ 的值。

$$C_{i,p} = \bigoplus_{j=0}^{p-1} D_{i,j} \quad 0 \leq i \leq p-2 \quad (1)$$



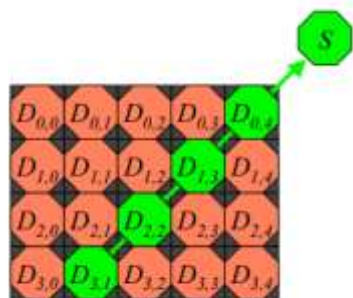
$$C_{0,0} = D_{0,0} \oplus D_{0,1} \oplus D_{0,2} \oplus D_{0,3} \oplus D_{0,4}$$

$$C_{1,0} = D_{1,0} \oplus D_{1,1} \oplus D_{1,2} \oplus D_{1,3} \oplus D_{1,4}$$

$$C_{2,0} = D_{2,0} \oplus D_{2,1} \oplus D_{2,2} \oplus D_{2,3} \oplus D_{2,4}$$

$$C_{3,0} = D_{3,0} \oplus D_{3,1} \oplus D_{3,2} \oplus D_{3,3} \oplus D_{3,4}$$

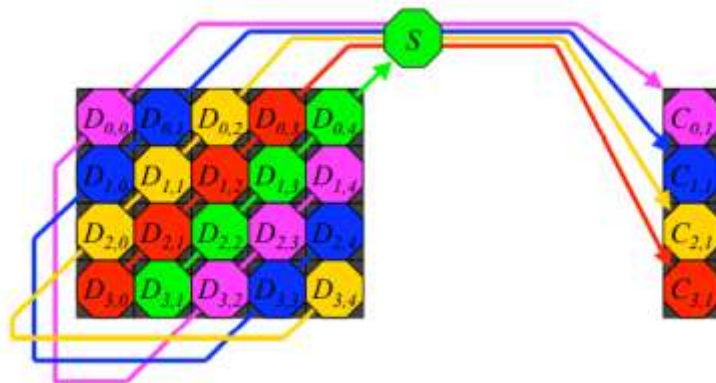
(2) 计算第二列校验信息即 $C_{0,1}$ 、 $C_{1,1}$ 、 $C_{2,1}$ 、 $C_{3,1}$ 的值。



$$S = \bigoplus_{j=1}^{p-1} D_{p-1-j,j} \quad (2)$$

$$C_{i,p+1} = S \oplus \bigoplus_{j=0}^{p-1} D_{j,<i-j>p} \quad 0 \leq i \leq p-2 \quad (3)$$

(记 $<n>p=j$ 为 $j=n \bmod p$, $0 \leq j \leq p+1$)



$$S = D_{3,1} \oplus D_{2,2} \oplus D_{1,3} \oplus D_{0,4}$$

$$C_{0,1} = D_{0,0} \oplus D_{3,2} \oplus D_{2,3} \oplus D_{1,4} \oplus S$$

$$C_{1,1} = D_{1,0} \oplus D_{0,1} \oplus D_{3,3} \oplus D_{2,4} \oplus S$$

$$C_{2,1} = D_{2,0} \oplus D_{1,1} \oplus D_{0,2} \oplus D_{3,4} \oplus S$$

$$C_{3,1} = D_{3,0} \oplus D_{2,1} \oplus D_{1,2} \oplus D_{0,3} \oplus S$$

在这里介绍的是**EVENODD双列删错的译码算法**，在译码过程中只需要简单的异或操作。

假定数据块*i*和*j*损坏， $0 \leq i < j \leq p+1$ ，假设数据块的排列从左到右是 $0 \sim p+1$ ，其中 $0 \sim p-1$ 列为用户数据块， p 、 $p+1$ 列为校验块。

(1) $i=p, j=p+1$, **两个检验数据块损坏**，数据恢复方式是重新编码。

(2) $i < p, j=p+1$, **一个用户数据块和一个校验块损坏**，恢复用户数据块*i*是通过参数*S*进行恢复，即

$$S = D_{\langle i-1 \rangle p, p+1} \oplus (\oplus_{l=0}^{p-1} D_{\langle i-l-1 \rangle p, l}) \quad (4)$$

假定 $D_{p-1, l} = 0, 0 \leq l \leq p$ ，然后利用下式计算*i*数据块的信息：

$$D_{k, i} = S \oplus D_{\langle i-1 \rangle p, p+1} \oplus (\oplus_{l=0, l \neq i}^{p-1} D_{\langle k+i-l \rangle p, l}), 0 \leq k \leq p-2 \quad (5)$$

(3) $i < p, j = p$, 一个用户数据块和一个校验块损坏, 用户数据块 i 可以根据平行校验数据块进行异或得到恢复, 即

$$D_{k,i} = \bigoplus_{l=0, l \neq k}^p D_{k,l}, \quad 0 \leq k \leq p-2 \quad (6)$$

(4) $i < p, j < p$, 两个用户数据块损坏, 数据恢复方法如下:

① 首先将最后两列校验数据块的信息异或, 恢复参数 S

$$S = (\bigoplus_{l=0}^{p-2} D_{l,p}) \oplus (\bigoplus_{l=0}^{p-2} D_{l,p+1}) \quad (7)$$

② 然后计算 $S^{(0)} = S_0^{(0)}, S_1^{(0)}, \dots, S_{p-1}^{(0)}, S^{(1)} = S_0^{(1)}, S_1^{(1)}, \dots, S_{p-1}^{(1)}$, 其中

$$S_u^{(0)} = \bigoplus_{l=0, l \neq i, j}^p D_{u,l} \quad (8)$$

$$S_u^{(1)} = S \oplus D_{u,p+1} (\bigoplus_{l=0, l \neq i, j}^{p-1} D_{<u-l>p,l}) \quad (9)$$

③ 然后按照下面的顺序恢复 i, j 数据块的各个信息:

- 1 Set $s \leftarrow \langle -(j-i)-1 \rangle_p$ and $a_{m-1,l} \leftarrow 0$ for $0 \leq l \leq p-1$
- 2 Let $a_{s,j} \leftarrow S_{\langle j+s \rangle_p}^{(1)} \oplus a_{\langle s+(j-i) \rangle_p, i}$ and $a_{s,i} \leftarrow S_s^{(0)} \oplus a_{s,j}$
- 3 Set $s \leftarrow \langle s-(j-i)-1 \rangle_p$ if $s=p-1$ then stop, else goto step 2

利用译码算法恢复第0和2个错误数据块的信息：



举个例子！

初始数据

?	0	?	1	0	1	1
?	1	?	0	0	0	1
?	1	?	0	0	1	1
?	1	?	1	1	0	0

初始数据

?	0	?	1	0	1	1
?	1	?	0	0	0	1
?	1	?	0	0	1	1
?	1	?	1	1	0	0

第一步：找到公共因子 S ，将这个数组的最后两列相异或，得到 $S=1$ 。

这意味着，对角校验是奇校验。根据公式(6)得到校验数组：

$$S^{(0)} = 0\ 1\ 0\ 1\ 0, S^{(1)} = 0\ 1\ 0\ 1\ 0$$

第二步：利用递归恢复丢失的数据 $a_{l,0}, a_{l,2}, 0 \leq l \leq 3$ 。

由于 $i = 0, j = 2$, 故 $s = \langle -(j-i)-1 \rangle_p = \langle -3 \rangle_5 = 2$ ，后续计算过程如表所示。

1	2	3	4
$S \leftarrow 2$	$a_{2,2} \leftarrow SD_4 = 0$	$a_{2,0} \leftarrow SH_2 \oplus a_{2,2} = 0$	$S \leftarrow 0$
$S \leftarrow 0$	$a_{0,2} \leftarrow SD_2 \oplus a_{2,0} = 0$	$a_{0,0} \leftarrow SH_0 \oplus a_{0,2} = 0$	$S \leftarrow 3$
$S \leftarrow 3$	$a_{3,2} \leftarrow SD_0 \oplus a_{0,0} = 0$	$a_{3,0} \leftarrow SH_3 \oplus a_{3,2} = 1$	$S \leftarrow 1$
$S \leftarrow 1$	$a_{1,2} \leftarrow SD_3 \oplus a_{3,0} = 0$	$a_{1,0} \leftarrow SH_1 \oplus a_{1,2} = 1$	$S \leftarrow 4$

由于 $S=4=p-1$,故算法停止。

定理一： EVENODD译码算法可以恢复任意2个数据块出错的数据。

证明：如果只有一列出错，我们可以利用剩余的正确校验位数据相异或进行恢复。根据异或的性质可以很容易知道这是完全可行的。

如果有两列出错，假设第 i 和 j 列出错， $0 \leq i < j \leq p-1$ 。我们假设译码算法的四种情况：

(1) $i=p, j=p+1$ ，这种情形和编码过程完全一样，显然此算法可行；

(2) $i < p, j=p+1$ ，注意公共因子 S 的校验值是由公式(4)得到的，它由对角线位 $(\langle i-1 \rangle p, 0), (\langle i-2 \rangle p, 1), \dots, (i, p-1)$ 上的数值相异或得到。这是一条唯一不与第 i 列相交的对角线(此时第 i 列无效)。因此所有的对角线都有相同的公共因子 S 。

经过分析公式(3)得到 $a_{k,i}$ 的过程, 我们可以推出能够找到恢复第 i 列数值的公式(5), 一旦第 i 列的值被恢复, 这个算法就可以利用公式(1)得到第 p 列的值。

(3) $i < p, j = p$, 这种情形和编码的过程相似, 在这里不再赘述。

(4) $i < j < p$, 第一步通过公式(7)计算出共同因子 S , 也就是公式(3)和公式(7)是等价的, 但是公式(3)中存在未知的变量 i, j 的值, 因此不能使用公式(3)直接得到共同因子 S , 因此根据公式(1)和(2)可以得到下面的式子(也就是公式(7)):

$$\begin{aligned} (\oplus_{l=0}^{p-2}) \oplus (\oplus_{l=0}^{p-2} a_{l,p+1}) &= (\oplus_{l=0}^{p-2} (\oplus_{t=0}^{p-1} a_{l,t})) \oplus (\oplus_{l=0}^{p-2} (S \oplus \oplus_{t=0}^{p-1} a_{<l-t>p,t})) \\ &= (\oplus_{l=0}^{p-2} (\oplus_{t=0}^{p-1} a_{l,t})) \oplus (p-1)S \oplus (\oplus_{l=0}^{p-2} (S \oplus \oplus_{t=0}^{p-1} a_{<l-t>p,t})) \\ &= (\oplus_{l=0}^{p-2} (\oplus_{t=0}^{p-1} a_{l,t})) \oplus (\oplus_{l=0}^{p-2} (\oplus_{t=0}^{p-1} a_{<l-t>p,t})) \end{aligned} \quad (10)$$

因为 $p-1$ 为偶数，所以 $(p-1)S \equiv 0 \pmod{2}$ ，又因为最后一个虚拟的 $p-1$ 行所有的值都为0，即 $a_{p-1,t}=0$ ，因此可以得到 $(\bigoplus_{l=0}^{p-2}(\bigoplus_{t=0}^{p-1} a_{l,t}))$ 所有水平校验列的值异或，也就是所有的0到 $p-1$ 列的数据(每列是 $p-1$ 行)，所有数据位异或。 $(\bigoplus_{l=0}^{p-2}(\bigoplus_{t=0}^{p-1} a_{<l-t>p,t}))$ 表示所有的垂直校验列的值异或(除掉共同因子 S)，从0到 $p-1$ (每列 p 行，但最后一行的值为0)，因此也就是所有的0到 $p-1$ 列所有数据位异或。

因此：

$$\begin{aligned} 0 &= (\bigoplus_{l=0}^{p-2}(\bigoplus_{t=0}^{p-1} a_{l,t})) \oplus (\bigoplus_{l=0}^{p-1}(\bigoplus_{t=0}^{p-1} a_{<l-t>p,t})) \\ &= (\bigoplus_{l=0}^{p-2}(\bigoplus_{t=0}^{p-1} a_{l,t})) \oplus (\bigoplus_{l=0}^{p-2}(\bigoplus_{t=0}^{p-1} a_{<l-t>p,t})) \oplus (\bigoplus_{t=1}^{p-1} a_{p-1-t,t}) \end{aligned}$$

所以得到如下公式：

$$(\bigoplus_{l=0}^{p-2}(\bigoplus_{t=0}^{p-1} a_{l,t})) \oplus (\bigoplus_{l=0}^{p-2}(\bigoplus_{t=0}^{p-1} a_{<l-t>p,t})) = \bigoplus_{t=1}^{p-1} a_{p-1-t,t} \quad (11)$$

公式(11)证明了公式(2)和公式(7)都能给出公共因子 S 的值。

下面给出证明“这个算法只能唯一恢复第*i*列和第*j*列的数据”。

假设校验组 $S^{(0)}$ 和对角校验组 $S^{(1)}$ 已经分别由公式(8)和公式(9)获得。

第一步给 S 赋初值 $S = \langle -(j-i)-1 \rangle_p$ 得到 S 。

第二步计算：

$$a_{\langle -(j-i)-1 \rangle_p, j} \leftarrow S_{\langle i-1 \rangle_p}^{(1)} \quad (12)$$

该公式是正确的，通过公式(3)和公式(9)可以得出。

如果首先计算出 $a_{\langle i-1-j \rangle_p, j}$ 的值，可以根据下面的公式(13)计算得到 $a_{\langle -(j-i)-1 \rangle_p, j}$ ：

$$a_{\langle -(j-i)-1 \rangle_p, i} \leftarrow S_{\langle -(j-i)+j-1 \rangle_p}^{(0)} \oplus a_{\langle -(j-i) \rangle_p, j} \quad (13)$$

公式(13)的正确性，根据公式(1)和公式(8)可以得到。

第三步，参数 S 的值设置为 $\langle -2(j-i)-1 \rangle_p$ ，返回第二步，根据算法可以得到：

$$a_{\langle -2(j-i)-1 \rangle_p, j} \leftarrow S_{\langle -2(j-i)+j-1 \rangle_p}^{(1)} \oplus a_{\langle -(j-i) \rangle_p, i} \quad (14)$$

$$a_{\langle -2(j-i)-1 \rangle_p, j} \leftarrow S_{\langle -2(j-i)-1 \rangle_p}^{(0)} \oplus a_{\langle -2(j-i)-1 \rangle_p, j} \quad (15)$$

□□□□□□□□奇偶校验码，是一种完全基于XOR运算的纠错码，它是一种条块宽度为 $w=1$ 的特殊水平码。最简单的奇偶校验码是单奇偶校验码：在一个条带中，只有唯一的一个校验条块，这个校验条块由所有用户数据条块的XOR运算之和得到。于是单奇偶校验的容错能力为1，其最典型的应用是RAID 5。

基于单奇偶校验码可以构造出容错能力更高的奇偶校验码，大致可分为两类：

- (1)根据几何结构构造的奇偶校验码，例如水平垂直奇偶校验码
- (2)根据图结构构造的奇偶校验码，例如基于Tanner图的LDPC码

根据几何结构构造的奇偶校验码。这类奇偶校验码将一个条带中的条块在逻辑上组织成多维几何结构，然后在每个方向上采用单奇偶校验码进行编码，因此具有十分规整的结构，十分易于实现。但是，他们的存储效率通常较低。

eg: 水平垂直奇偶校验码

编码方式：他将一个条带中的条块在逻辑上组织成二维阵列结构，然后在水平和垂直方向上分别采取单奇偶校验码进行编码。

$D_{0,0}$	$D_{0,1}$	$\cdots$	$D_{0,(k_2-1)}$	$\rightarrow$	$P_{0,*}$
$D_{1,0}$	$D_{1,1}$	$\cdots$	$D_{1,(k_2-1)}$	$\rightarrow$	$P_{1,*}$
$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\rightarrow$	$\vdots$
$D_{(k_1-1),0}$	$D_{(k_1-1),1}$	$\cdots$	$D_{(k_1-1),(k_2-1)}$	$\rightarrow$	$P_{(k_1-1),*}$
$\downarrow$	$\downarrow$	$\cdots$	$\downarrow$	$\searrow$	
$P_{*,0}$	$P_{*,1}$	$\cdots$	$P_{*,(k_2-1)}$		$P_{*,*}$

$D_{i,j}$ 表示用户数据条块

$P_{i,*}$ 表示校验条块

$$P_{i,*} = \sum_{j=0}^{K_2-1} D_{i,j}$$

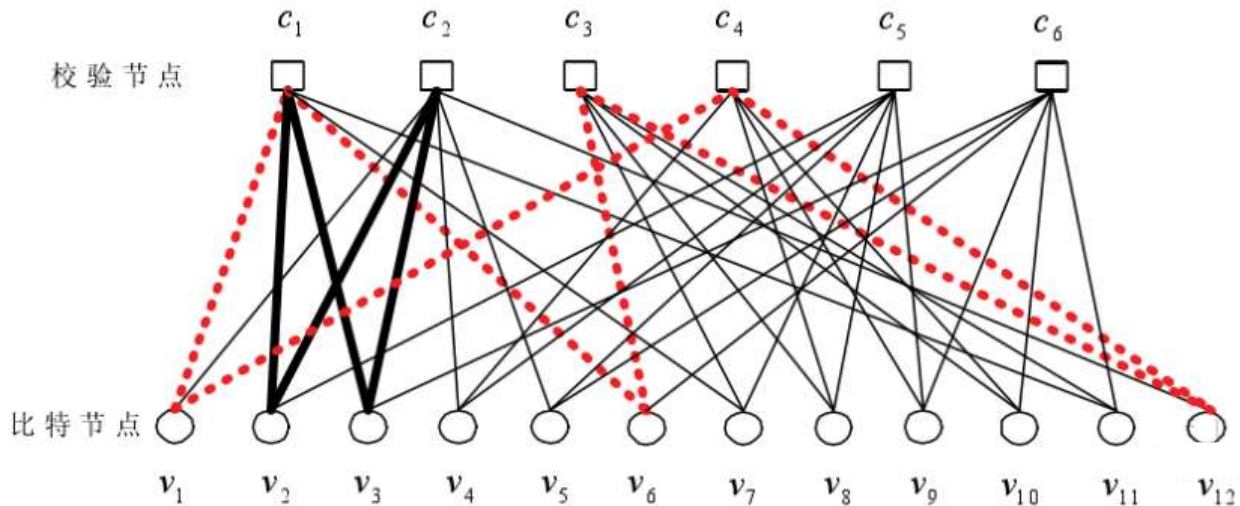
$$P_{*,j} = \sum_{i=0}^{K_1-1} D_{i,j}$$

$$P_{*,j} = \sum_{i=0}^{K_1-1} \sum_{j=0}^{K_2-1} D_{i,j}$$

$$(0 \leq i \leq K_1-1, 0 \leq j \leq K_2-1)$$

低密度奇偶校验码(Low-Density Parity-Check Code, LDPC)最初是为通信链路的容错而设计的，近年来开始在基于广域网的存储应用中被研究。LDPC码的主要优点在于，构造出的高容错LDPC码可以具有接近最高的存储效率并且解码复杂度很低。

LDPC码是一种由级联的随机稀疏二部图和一个传统的纠删码构成的特殊的纠删码，级联二部图的意思就是多级二部图。



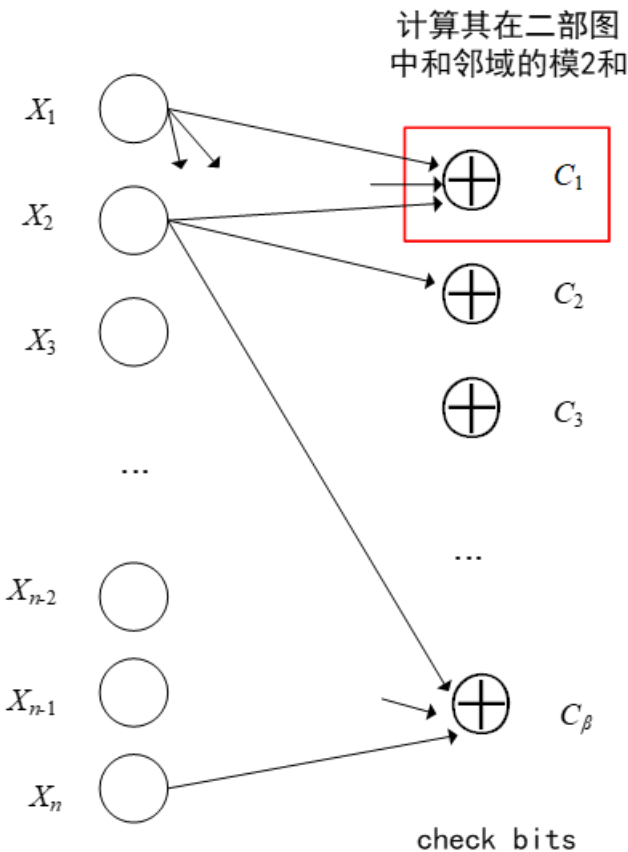
LDPC码示例

低密度奇偶校验码的一个典型的例子是龙卷风码(Tornado Code)

编码方式:

龙卷风码通过层层递进的异或操作得到校验码。

如图所示，把原数据分成n个等长的数据块($x_1, x_2, x_3, \dots, x_n$)，这些数据被称为用户数据块(message bits)。根据多级二部图 $B_0, B_1, \dots, B_k$ ， B_{k+1} 的连接关系，将相连的数据节点进行异或得到下一层校验节点(check bits)的值($C_1, C_2, \dots, C_\beta$)。



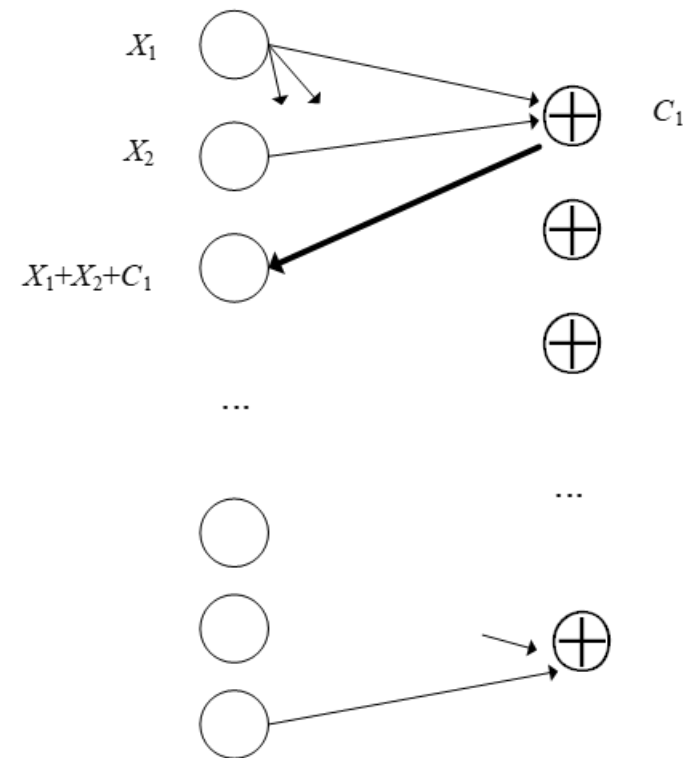
龙卷风码编码示例

举例来说，在右图中，某校验块层各节点的值分别为 $X_1, X_2, X_1+X_2+C_1, \dots$ ，下一级第一个校验块由 $X_1 \oplus X_2$ 得到。

选择哪些变量来进行异或操作由二部图确定，一般来说，级联低密度删码二部图的生成具有随机性，通过使用每一级二部图的连接关系，就可以依次生成级联二部图的所有右节点。

在此过程中，进行异或操作的次数(或者说是运算层数)由每层中的数据块数目决定。

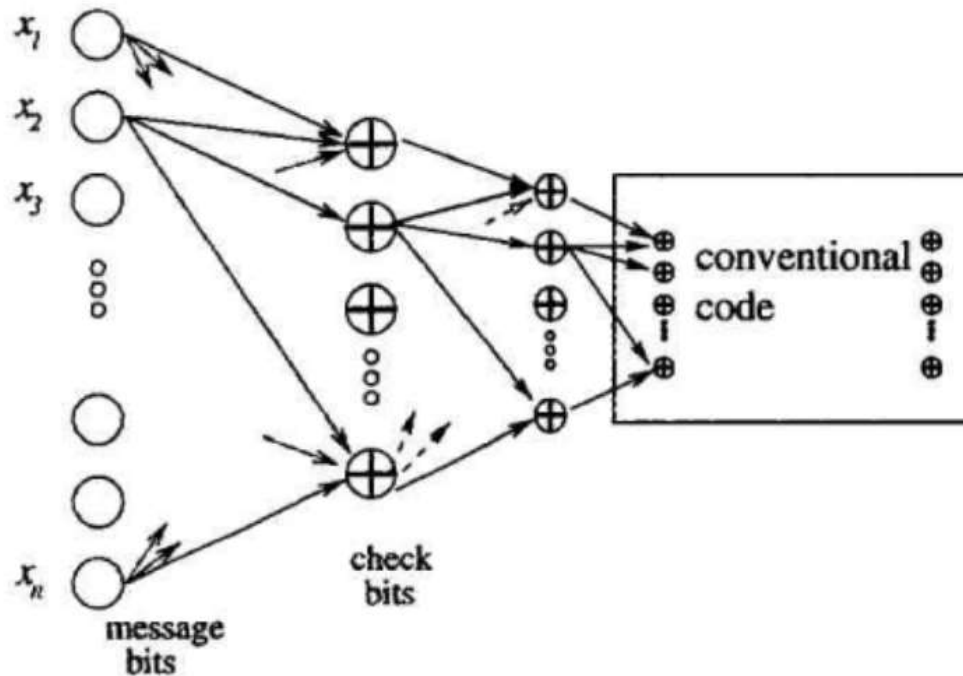
不妨引入一个参数 e , $e = m/(m+n)$ ，其中 m 是最终的校验码数目，也就是说， e 是擦除概率，它与码率的和为1。显然， e 满足 $e < 1$ 。那么，在校验块计算时，第一层得到的数据块数目就是 $e \times n$ ，第二层的数目是 $e^2 \times n$ ，第三层的数目是 $e^3 \times n$ ，依次递推，第 k 层的数据块数目就是 $e^k \times n$ 。



龙卷风码中校验块的计算过程

下图说明，**最后一级**(也就是二部图中的 B_{k+1})**的计算使用传统的编码方法**(如RS码的编码方法)。最后一级的运算中， B_{k+1} 和 B_k 具有相同的变量节点，根据连接关系生成新节点。

至此，整个编码工作完成。发送方传输的编码就是原数据包加上校验块的总数据包。



龙卷风码解码过程

解码方式:

接收方收到 $n+m$ 个数据包, 如果包含源数据的 n 个数据包有丢失, 就可以通过 m 个冗余数据包进行重构。

解码过程实际上是一个在二部图上去边的过程, 从最高级二部图开始:

- (1)首先, 利用 B_k 和 B_{k+1} 得到它们的信息块, 即 B_{k-1} 的校验块。
- (2)然后, 依次得到 $B_{k-2}, \dots, B_1, B_0$ 的校验块。
- (3)最后, 得到 B_0 的 m 个信息块。

当然, 如果在解码开始时就已经得到了某一层 $B_i(0 \leq i < k)$ 的所有校验块, 那么就可以直接从 B_i 开始解码。

容错性：

如果与某一校验节点相邻的信息节点中存在一个丢失数据的信息节点，那么可以通过该校验节点和其它节点进行异或运算，得到丢失的信息数据。

数据恢复：

这里的“信息节点”与“校验节点”是**相对的**，在每一层异或运算中，参数节点称为“信息节点”，生成节点称为“校验节点”。具体步骤如下：

(1)所有校验节点赋初始值0，所有信息节点根据接收的信息赋值为0,1,或D (删除)。

(2)根据边的连接关系，**把所有未被删除的信息节点的值和与其相连的校验节点的值(模二)相加**，并将得到的值作为该校验节点的新值，然后从二部图中去掉这些正确接收的变量节点以及与之相连的边。

(3)在得到的二部图中寻找度数为1的校验节点。如果某一校验节点度数为1,则该校验节点的值就等于唯一与它相连的信息节点的值,此信息节点所对应的信息比特得到恢复。

(4)再根据边的连接关系,把恢复出来的信息节点的值与和它相连的校验节点的值(模二)相加,作为这些校验节点的新值,并从二部图中去掉恢复出来的信息节点和与之相连的边。

(5) 在去边之后的二部图中重复步骤(3)(4), 直至二部图中不存在度数为1的校验节点, 或所有的原数据(message bits) 值均被恢复。

这样, 所有能恢复的丢失数据都能被恢复。此解码过程中只有异或计算, 故解码速度较其他纠错码快速。

龙卷风码的复杂度分析	Tornado code
编码时间复杂度	$O(n \ln(1/e))$
解码时间复杂度	$O(n \ln(1/e))$
编码率	$1 - p(e) \exp(1+e)$

优点:

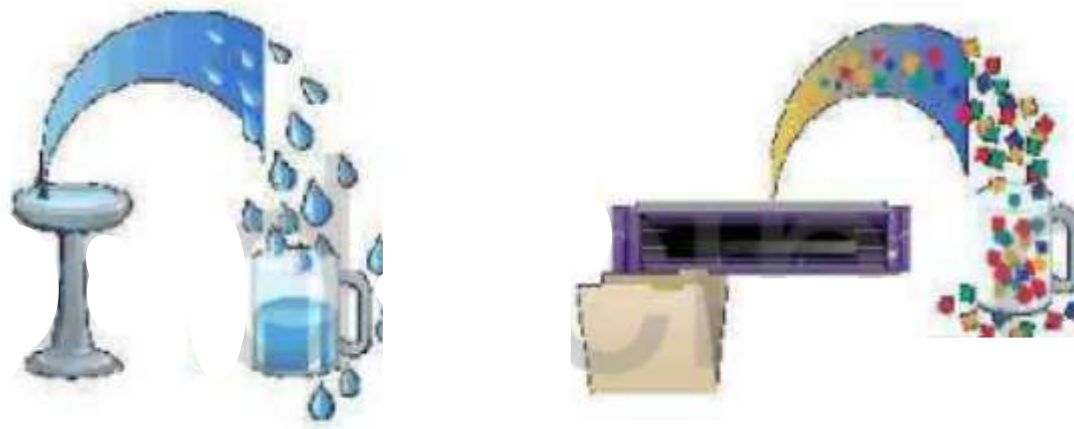
- (1)由于只涉及异或操作, 编码率高, 其中 $p(e)$ (e 是擦除概率)是一个极小量, 与二部图的选择有关。
- (2)编码过程提到, 进行异或操作的次数, 也就是二分图的边的数目, 由每层中的数据块数目决定。因此该算法的复杂度为线性, 编解码效率高。
- (3)由于所有的丢失数据基本能恢复, 龙卷风码能提供接近最优的损失保护。

缺点:

- (1)需要擦除概率 e 作为输入, 而擦除概率难以精确获得、随时可能变化、多个接受者的擦除概率不同的特点也带来了这种算法的缺点。
- (2)和RS Code一样, 在数据规模较大, 且在数据规模较大的情况下, 编解码优势才比较明显。数据规模较小时并不能体现很大优势。

数字喷泉码是一类码率不受限制的纠删码(Rateless Erasure Code), 也称为无速率编码, 是一类基于图的线性纠删码。喷泉码是指可以由 k 个原始数据分组生成任意数量的编码分组, 而接收方只要收到其中任意 m 个编码分组, 即可通过译码以高概率成功恢复全部原始数据分组。

可以看到, 上述编码过程就如同源源不断产生水滴(编码分组)的喷泉(编码器), 而我们只要用杯子(译码器)接收足够数量的水滴, 即可达到饮用(成功译码)的目的。正因如此, 该种编码被称为喷泉码。



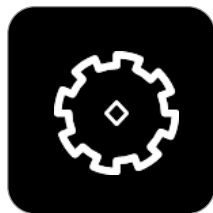
数字喷泉码的形象说明



John Byers 及Michael Luby 等人于1998 年首次提出了数字喷泉(Digital Fountain)的概念，它是针对大规模数据分发和可靠广播的应用特点而提出的一种理想的解决方案。但当时并未给出实用数字喷泉码设计方案。



2002年，M.Luby提出了第一种现实可行的喷泉码——LT码。在学术理论日渐完善的同时，喷泉码也日益受到产业界的关注，获得了越来越多的实际应用。

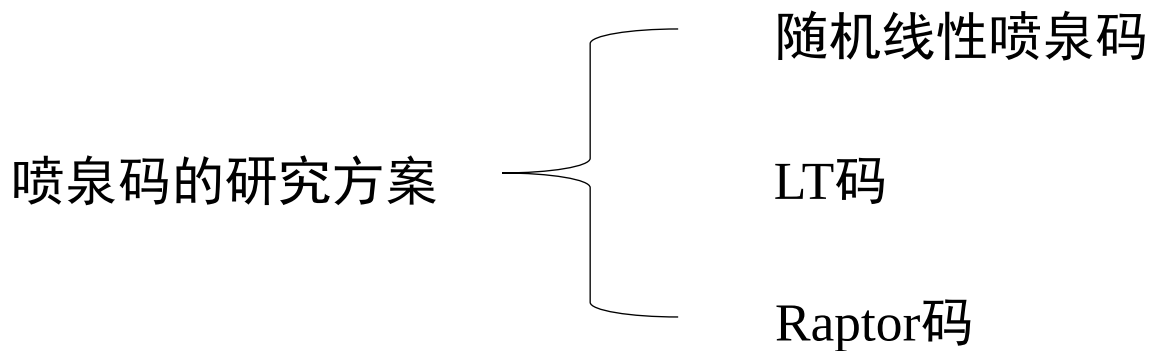


目前，一种由Digital Fountain公司设计的系统Raptor码也已经被DVB-H标准和3GPP组织的MBMS标准采用，并且正在参与其他多项国际标准的制定。

喷泉码的研究现状：

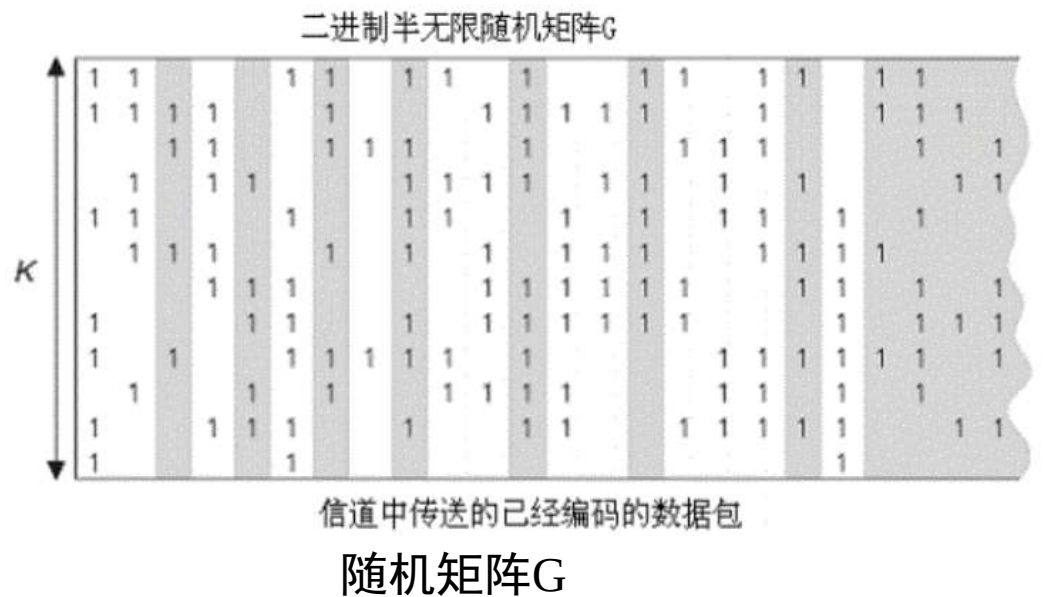
(1)应该尽量减小译码开销 ε ,使其趋于零。

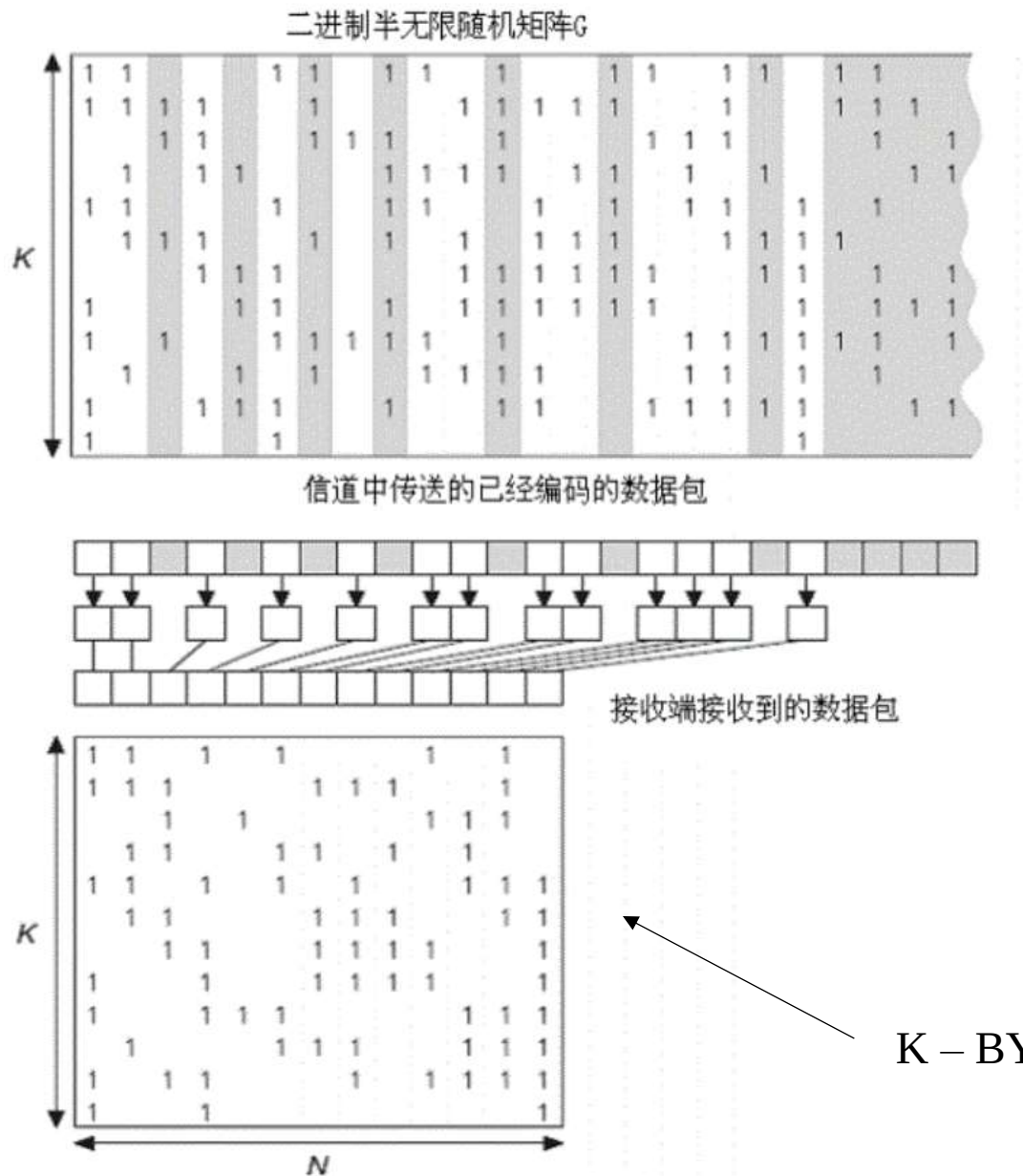
(2)应该尽量减小编译码复杂度，理想情况下，应该使生成每个编码分组需要的运算量是一个与 k 无关的常数，而成功译码 m 个编码分组获得 k 个原始数据分组需要的运算量是一个关于 k 的线性函数。



假设编码器需要编码的文件为 $S_1, S_2, \dots, S_k$ ，每个 S_k 都是一个信源符号，它只能被完整的传输或者被删除。假定每个 S_k 大小都是相同的比特数。在每一个时钟周期，编码器产生一个 K 阶随机二进制矩阵 $\{G_{kn}\}$ ，编码器的输出 t_n 由 S_k 异或而成，**参加运算的 S_k 对应的 $G_{kn}=1$** ，即 $t_n = \sum_{k=1}^K S_k G_{kn}$

这个和可连续的由不同的 S_k 异或而来，可以**对每 K 个随机 t_n 在一个半无限的二进制矩阵中定义一个新的列**，如图所示。





经过信道传输，发生数据包删除，接收端一直接收数据包，直到接收到N个数据包。

$K - BY - N$ 矩阵，称其中一部分为G

(1) 如果 $N < K$ ，接收端就没有接收到足够多的码元，不可能恢复源文件。

- 如果 $N = K$ ，则接收端有可能恢复源文件。如果矩阵 G 可逆（二进制），就可以计算出 G 的逆矩阵 G^{-1} ，就能恢复 S_k ，即 $S_k = \sum_{n=1}^N t_n G_{nk}^{-1}$ ，此时数据恢复的概率为 $(1-2^{-K})(1-2^{-(K-1)}) \dots (1-1/8)(1-1/4)(1-1/2)$ ，当 $K > 10$ 时，概率约为0.289。
- 如果 $N > K$ ，令 $N = K + E$ ， E 是多接收到的很小量的码元。令此时数据恢复的概率为 $1-e$ ， e 为接收端接收到额外的码元时无法恢复的概率。对于任何 K ，无法恢复源文件的概率 $e(E) \leq 2^{-E}$ 。总的来说，恢复源文件的概率为 $1-e$ 时，需要接收的数据包数目大概是 $K + \log_2(1/e)$ ，对一个数据包编码的代价是需要 $K/2$ 次数据包运算，解码的代价是对随机矩阵 G 求逆的代价(大概 K^3 次二进制运算)和对接收的数据包对应的随机矩阵求逆的代价(大概是 $K^2/2$ 次数据包运算)之和。

当一个随机码对删除信道而言并不是技术上的最佳码，在接收端接收到 K 个数据包时，只有0.289的概率恢复源文件，这几乎可以认为是最好的了。但当接收到额外的 E 个数据包时，无差错的恢复源文件的概率就至少为 $1-e$ ，其中 $e = 1 - 2^{-E}$ 。因此，随着源文件大小的增加，随机线性喷泉码能够无限地接近香农理论的极限。唯一的缺点是随机线性喷泉码的编码和解码的代价都是二次方，编码时还会出现三次方。当 K 比较小时，这并不重要，但若 K 很大时，编码和解码的代价就会很大。因此，我们希望更小的编码和解码代价来解决问题。

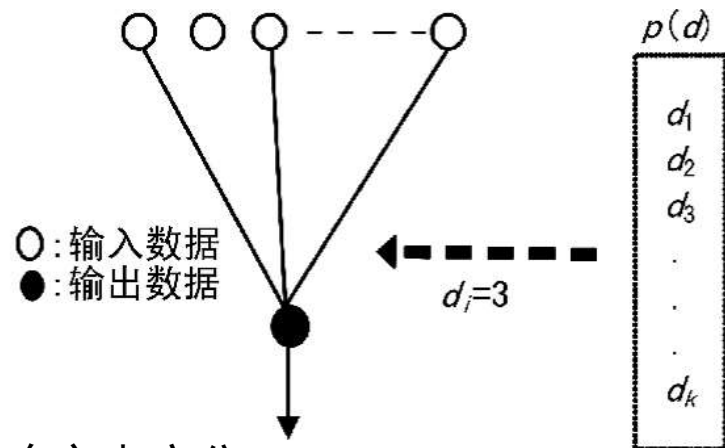
将长为 n 的文件分割成 $k=n/w$ 个等长的输入符号 s_k (每个输入符号的长为 w)。每个数据包称为输入符号，每个编码包为输出符号。若一个输出符号结点的度为 d ,定义LT码的度分布 $p(d)(d \geq 1)$ 为 d 的概率分布。编码步骤如下：

(1)编码器首先在 $1 \sim k$ 范围内按某一分布 p (称为编码度分布)**随机选取一个整数 d** ,其中 k 称为该码的码长， d 称为编码分组的度；

(2)**在所有输入符号中随机均匀地选取 d 个符号**作为该编码符号的邻节点符号；

(3)**将 d 个数据进行异或并输出**，即对这 d 个输入符号求异或得到一个编码符号 t_n 。

不断重复上面的步骤，就可以得到无限多个编码分组。



在这些步骤中一些细节需要注意：**LT码编码之前要先确定度分布函数**，度值的确定由度分布函数来计算。不同的度分布， k 的概率值是不同的，但**度分布概率函数只是确定了取度值为 d 的概率是多少，而并不能确定这 d 个原始编码分组是哪些，所以对于这 d 个原始编码分组的选择来说是任意的**，这里对这 d 个原始编码分组的选择采用均匀分布来进行选取，即从区间 $[1,k]$ 中按均匀分布随机生成 d 个整数。区间 $[1,k]$ 中所有整数被选取的概率是相等的。把这 d 个整数进行异或，就得到了一个编码分组。

随机度 d 的确定(孤波分布):

$$\rho(d) = \begin{cases} \frac{1}{k} & d = 1 \\ \frac{1}{d(d-1)} & d = 2, 3, \dots, k \end{cases} \quad (1)$$

根据度分布函数来确定编码时的度(参与编码的源文件的个数)。

假设 $k=100$, 则由孤波分布可知:

度为1的概率为 $1/100$, 度为2的概率为 $1/2$, ..., d 为100的概率为 $1/9900$ 。

在 $[1, 10000]$ 区间上划分100个区间:

度	取该度的概率	取该度的空间
1	$1/100$	$(0, 100]$
2	$1/2$	$(100, 5100]$
3	$1/6$	$(5100, 6766]$
...	...	...
100	$1/9900$	$(9999, 10000]$

确定了一次编码的度后, 还需要确定是由哪几个 s_k 异或编码, 可以用和确定度相同的方法来选取 s_k 。

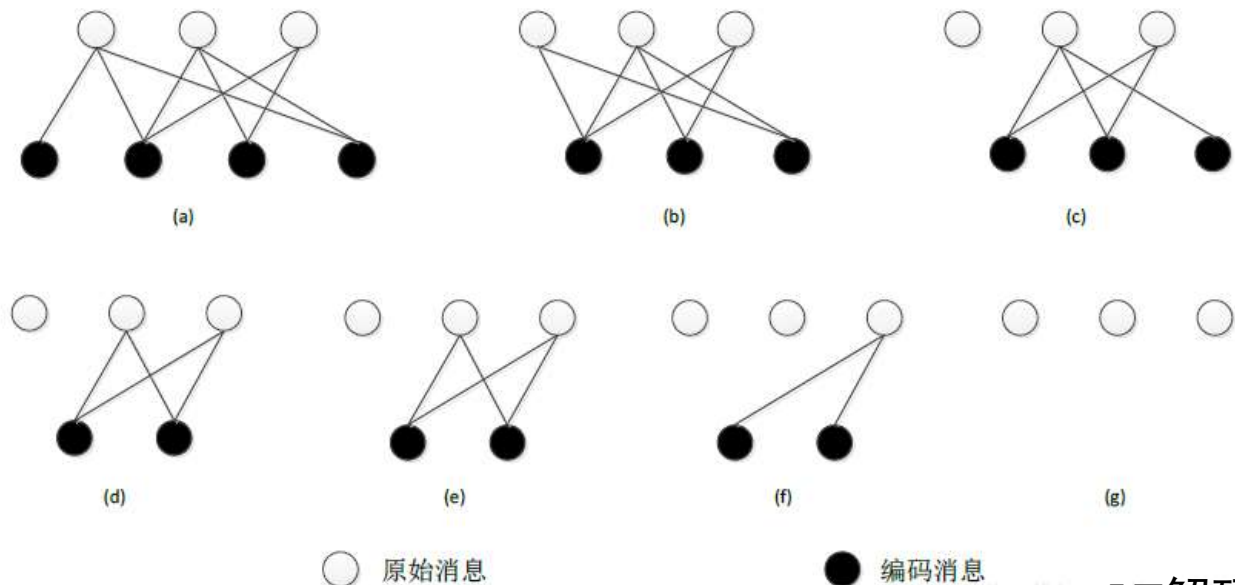
在 $[1, 10000]$ 区间上划分100个区间:

s_k	取该度的概率	取该度的空间
S_1	$1/100$	$(0, 100]$
S_2	$1/100$	$(100, 200]$
S_3	$1/100$	$(200, 300]$
...	...	...
S_{100}	$1/100$	$(9900, 10000]$

在0到10000之间随机产生 d 个数, 看这 d 个数落在哪个区间, 选取这些区间所对应的 s_k 进行编码。

LT码的解码采取**迭代法(BP算法)**。在解码的每一步，解码器都在**编码符号集合中**寻找**度为1的符号**，从而直接恢复出它们所连接的**输入符号**，然后将这些输入符号与跟它们有连接的编码符号进行异或并将结果取代该编码符号原来的值，完成之后删去它们之间的连接关系，并相应地修改编码符号的度。重复上述过程直至不存在度为1的符号为止。

如果所有输入符号都被恢复则解码成功，否则解码失败。



LT解码过程

具体步骤是：

(1)如图(a)所示，接收一定数量的编码分组，根据编码分组间的对应关系(可以通过分组头部等方式显式传递，也可以通过事先约定伪随机序列等方式隐式传递)建立双向图。顶层节点代表原始分组，底层节点代表编码分组，连接它们的边代表该原始分组是该编码分组的模二和分量。

(2)任意选取一个度数为1的编码分组。如果不存在，则译码停止;如果存在，则通过简单的复制运算，即可恢复与之相连的唯一原始分组，如图(b)所示。

(3)将已经恢复的原始分组模二和到与其相连的所有其他编码分组中，消除其在这些编码分组中的模二和分量。相应地，将双向图中对应的边删除，使得这些编码分组的度数减1。如图 (c)所示。如果某个编码分组的度数减小为1，则称该编码分组被“释放”，如图(c)最右侧的编码分组。

(4)重复步骤(2)和(3)，直至译码停止。如果所有原始分组都已经恢复，则译码成功;否则，译码失败，必须接收更多的编码分组才能继续译码。

一般来说，为了保证所有数据均能恢复出来，度的选择期望值必须高于 $O(\log n)$,舍弃编码过程中关于计算量的考虑。

	LT码
编码时间复杂度	$O(n \log n)$
解码时间复杂度	$O(n \log n)$
编码率	低

LT码的编解码时间复杂度均为 $O(n \log n)$ 。另外，LT码因为随机性，编码率并不确定，不过确定的是其编码效率较为低下。

产生原因：

LT码生成的编码包中有少量连接度很高的包，这些高连接度包的存在消耗了很多编译码异或操作，同时也降低了低连接度包的比例，从而减小了译码过程中可译集的大小，降低了译码成功率。

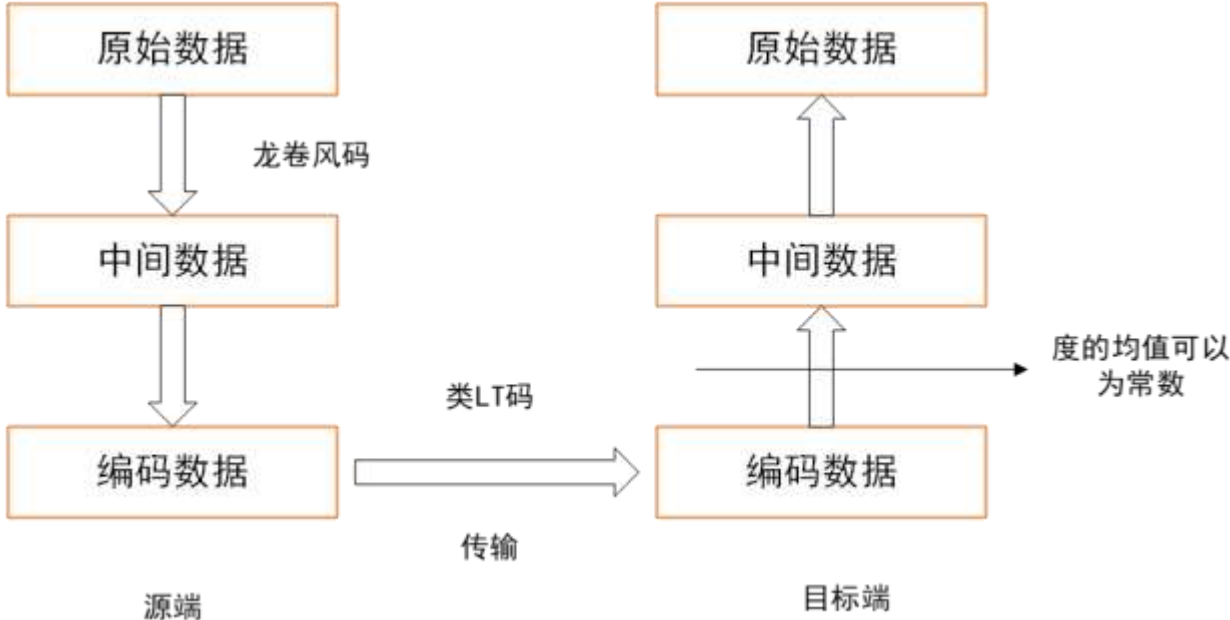
基本思想：

Raptor码提出采用两步编码的方式。首先对原始信息用一个分组码进行预编码，然后采用一个弱化的LT码对数据进行编码并发送。

课堂思政

Raptor码是在LT码的基础上发展出来的改进版，Raptor码的出现告诉我们有问题不可怕，改正就是了。正如党的十八大以来习总书记反复强调的“直面问题、解决问题”的思想，并且敢于直面问题、勇于修正错误使我们党的显著特点和优势。

如下图所示，通过**预编码**过程，首先采用某种传统的编码方法(图中采用龙卷风码编码方法，如同Raptor码的命名Rapid-Tornado所暗示，这里**一般采用的就是龙卷风码**)，将原始数据编码成为中间编码校验单元。然后将这些中间编码校验单元作为LT码的输入符号，进行编码。



Raptor码编解码过程

与编码过程相对应地，解码时只需要利用LT码解码技术恢复所有的中间编码校验单元，再根据中间编码的获得方式，使用传统编码的解码方式得到源数据包。

Raptor码的**编码和译码均需**
 $O(k \ln(1/\epsilon))$ 次包“异或操作”，
其中 ϵ 是译码开销。

Raptor码的**编译码复杂度与**
码长成线性关系，优于LT码的对
数关系。

Raptor 使用联合优化弱化
LT码和预编码的码率和参数可
以获得**更低的编译码复杂度**。

Raptor 使用联合优化弱化
LT码和预编码的码率和参数可
以在**相同译码开销下获得更高**
的译码成功率。

替代RAID

RAID的目的是在硬盘驱动器(HDD)失效时保护数据，它里面的数据可通过奇偶校验或镜像副本重新创建，但电子机械式硬盘发生故障的几率很高。Erasure code 存储消除了所有RAID问题。2009年Google在第二代GFS中使用了RS(6,3)编码，Facebook早期也在HDFS中使用了RS(10,4)编码。

各大互联网公司的存储服务

例如Google在GFS中使用了RS(6,3)编码，Facebook在HDFS中使用了RS(10,4)编码，Microsoft在Azure中使用了LRC(12,2,2)编码，阿里云在盘古中使用了RS(8,3)编码。

应用在AWCloud中

AWCloud中利用FPGA技术提高了纠删码的性能。

- 从数据类型的角度而言，**归档数据适合使用纠删码**。首先，归档数据需要保存相当长的一段时间，一份、两份或多份数据出现问题的可能性会急剧上升；其次，归档数据的写入操作很少，使得纠删码可以很轻易的恢复数据，并且纠删码的额外开销很少，如果没有故障发生，纠删码对读取操作通常不会有任何影响。
- 从应用的角度来看，对于读写都很繁忙的应用，使用纠删码并不是一个很好的选择。但对于**只有大量读取的应用**，纠删码可以提供强大的可靠性，并且在恢复数据的时候只产生很小的额外开销，因为纠删码的额外开销主要来自写操作。

——来自TechTarget中国原创内容中Miller的回答

- 评判纠删码的**第一条标准是性能**：即部署了一种纠删码后写入速度有多快，进行数据恢复时的读取速度有多快。
- 另一个标准是**纠删码如何将数据单元分割**：可以是整盘的分割，可以是卷级的分割，也可以是部分盘或部分卷的分割等等。**分割时还会遇到的一个问题是纠删码单元的数量有多少**，有10个数据单元和5个纠删码单元组成的纠删码系统，也有12个数据单元和4个纠删码单元组成的纠删码系统。但大部分纠删码产品都能配置数据单元和纠删码单元的个数。
- 最后需要考虑的是**某些纠删码类型会带来一些问题**：比如在部署之前需要了解每个厂商的纠删码是如何工作的，它们能在什么样的场景恢复数据故障。

——来自TechTarget中国原创内容中Miller的回答

- 在某些领域，纠删码会在未来掀起一波浪潮，特别是在**存储归档领域****纠删码是必备的**。因为当把数据保存10年甚至20年时会经历各种故障并将它们恢复，并且在数据归档中，需要经常读取数据，但不会产生额外的性能开销。
- 此外，未来纠删码是否会成为潮流**取决于设备容量大小和读写速率的相对速度提升差异**。当存储容量的增速大于读写速率的增速时，纠删码的作用就更能够体现，这就是为什么从90年代中期的RAID5走到了今天的RAID6。

——来自TechTarget中国原创内容中Miller的回答

灵活的设定副本数量，从而在数据持久性和成本间做出权衡，满足不同的应用场景。比如储存冷数据时，可将副本数缩小至1.2，即冗余数据是有0.2。

Erase code有内置的数据安全性。Erase code使用的数据块可以分布在城市、地区、一个数据中心或全球任何地方的不同存储位置。

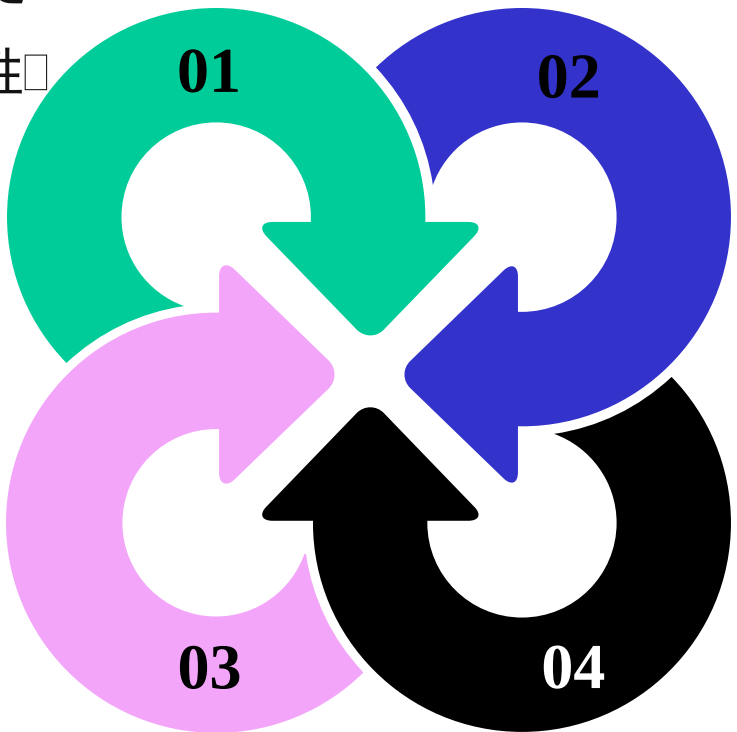
Erase code在面对自然灾害或技术故障时具有很好的恢复能力。另外，Erase code允许同时发生多种故障。





纠删码技术提高存储资源利用率的表现□：提供近似三副本的可靠性□

纠删码技术提高存储资源利用率的表现□：减小冗余存储□



纠删码技术的代价1：□□□计算量□

纠删码技术的代价□：网络负载□

EC码的优化——LRC